21CSC202J - OPERATING SYSTEMS

LAB MANUAL



II YEAR / III Semester

Academic Year: 2026-2027 ODD SEMESTER

FACULTY OF ENGINEERING AND TECHNOLOGY SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM CHENNAI- 600 089

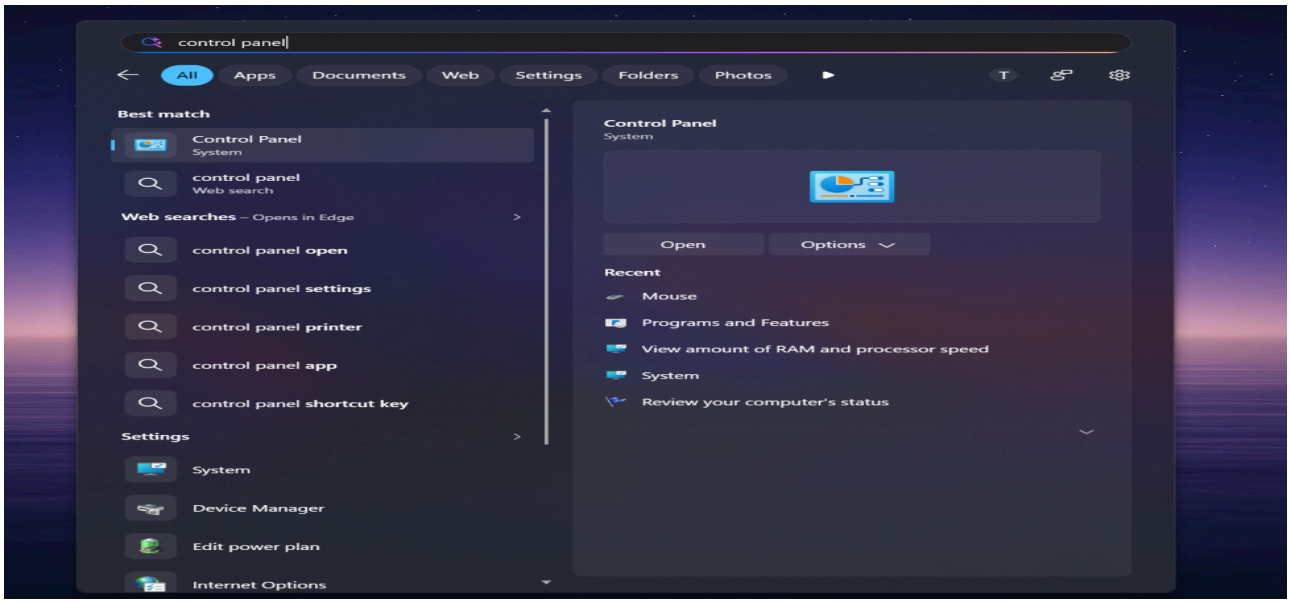
LIST OF EXPERIMENTS

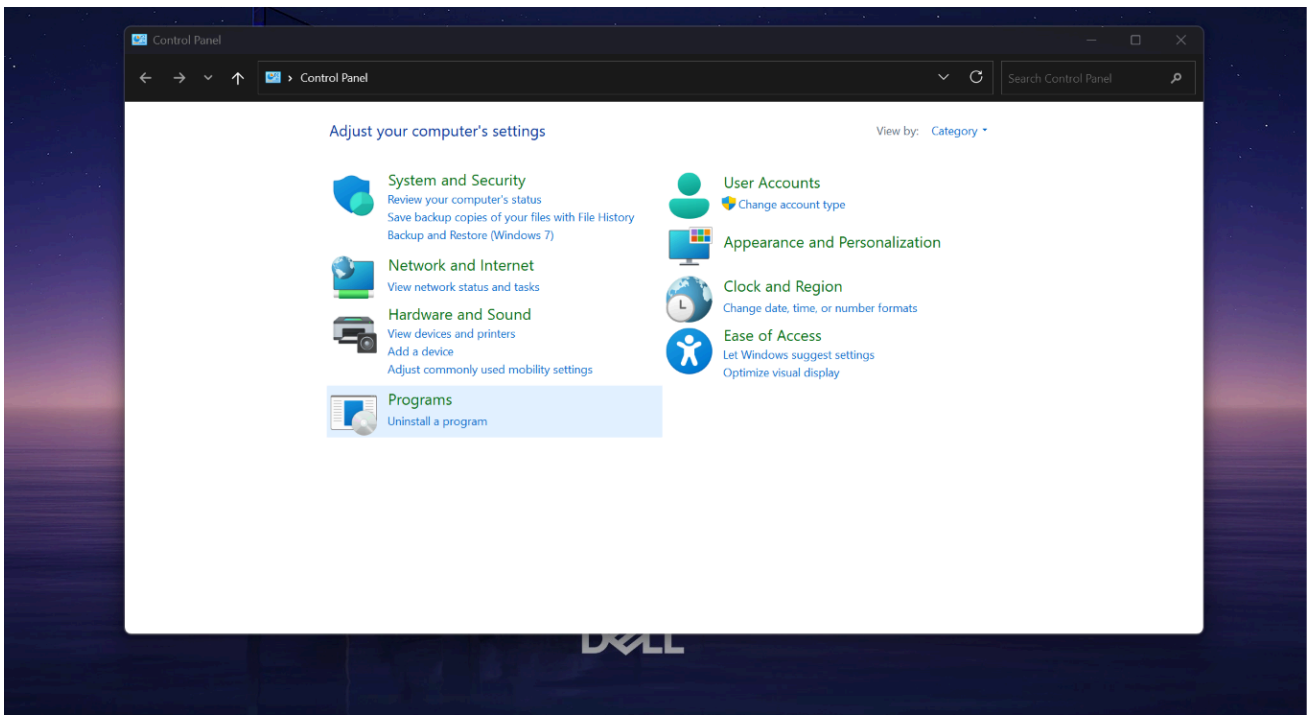
Ex.No.	Experiment Name
1	A. Operating system Installation, B. Basic Linux commands, C.Filter and admin commands
2	Process Creation using fork() and Usage of getpid(), getppid(), wait() functions
3	Multithreading
4	Mutual Exclusion using semaphore and monitor
5	Reader-Writer problem
6	Dining Philosopher problem
7	Bankers Algorithm for Deadlock avoidance
8	FCFS and SJF Scheduling
9	Priority and Round robin scheduling
10	FIFO Page Replacement Algorithm
11	LRU and LFU Page Replacement Algorithm
12	Best fit and Worst fit memory management policies
13	Disk Scheduling algorithm
14	Sequential and Indexed file Allocation
15	File organization schemes for single level and two level directory

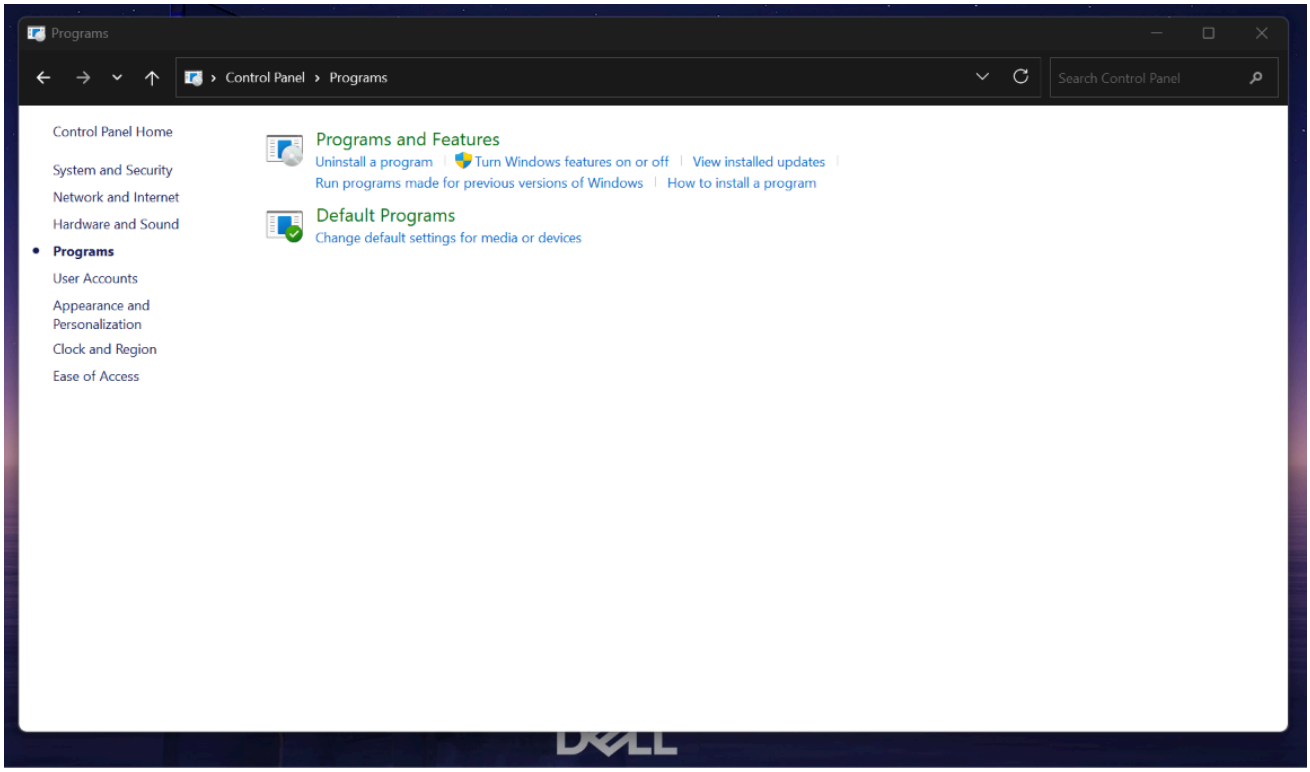
EXPERIMENT 1A :Operating System Installation

Ubuntu installation guidelines in windows

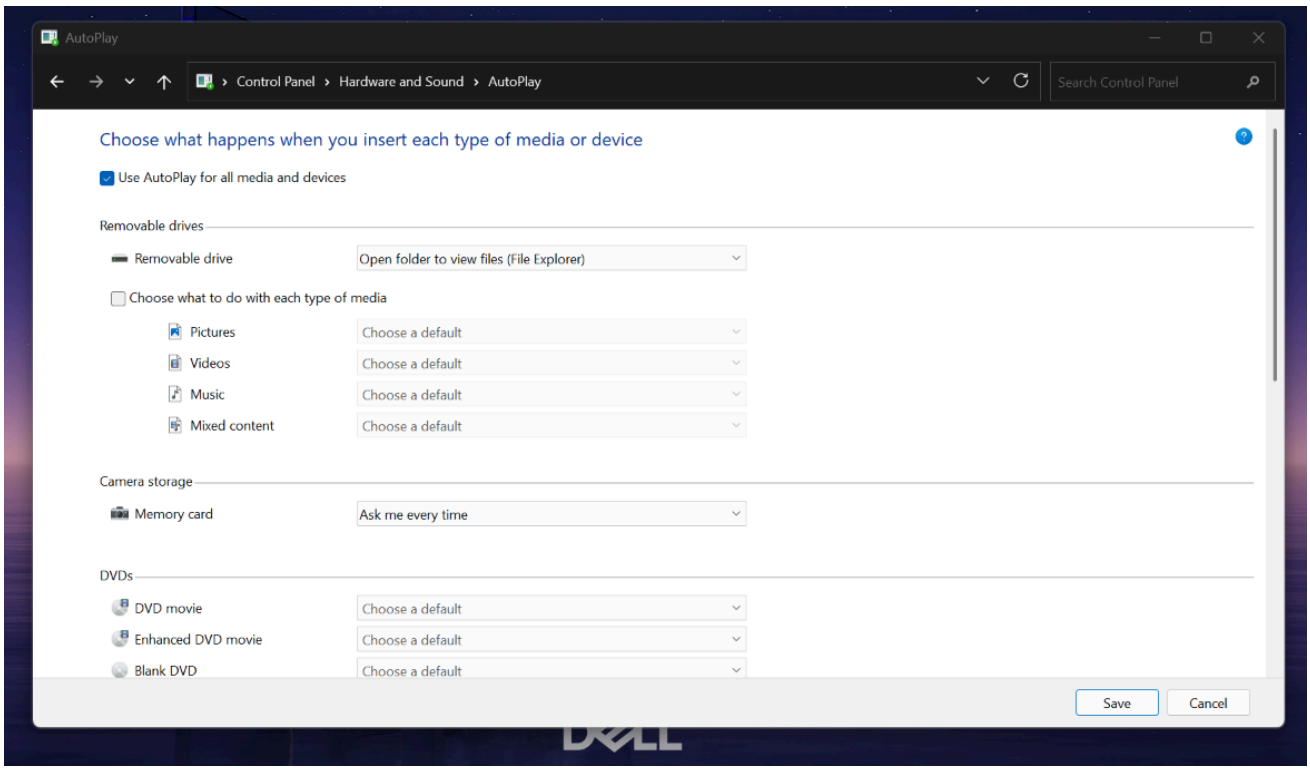
Installing Ubuntu in windows 10- 64 bit

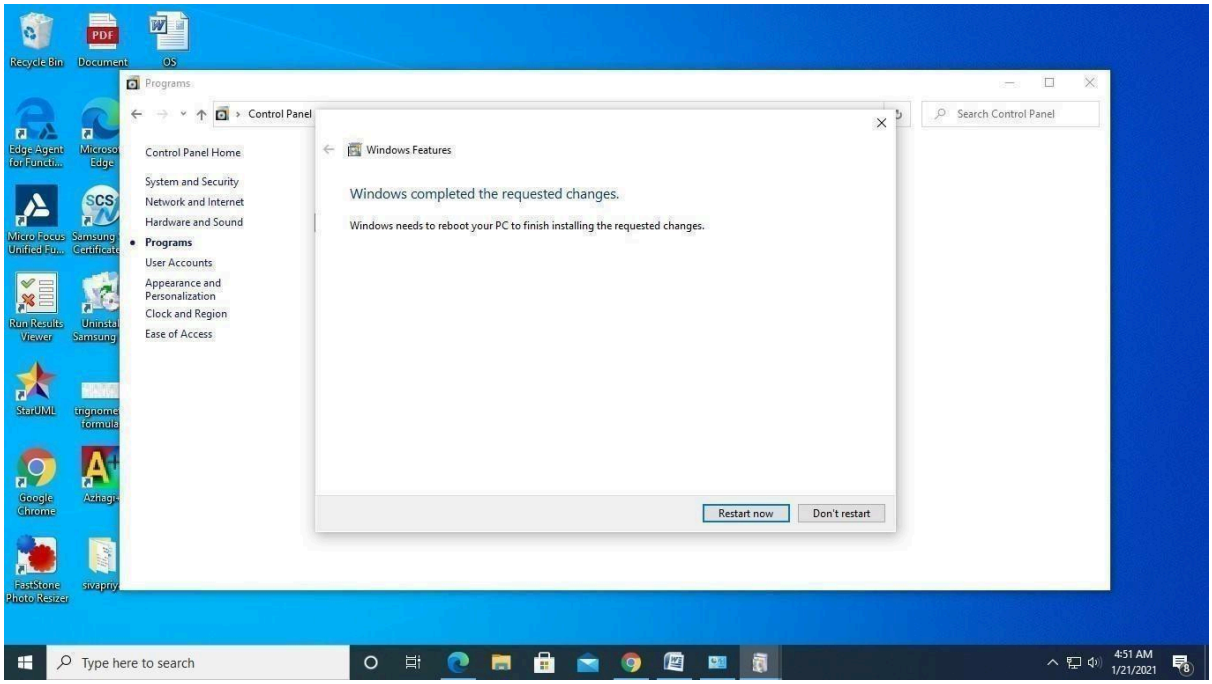


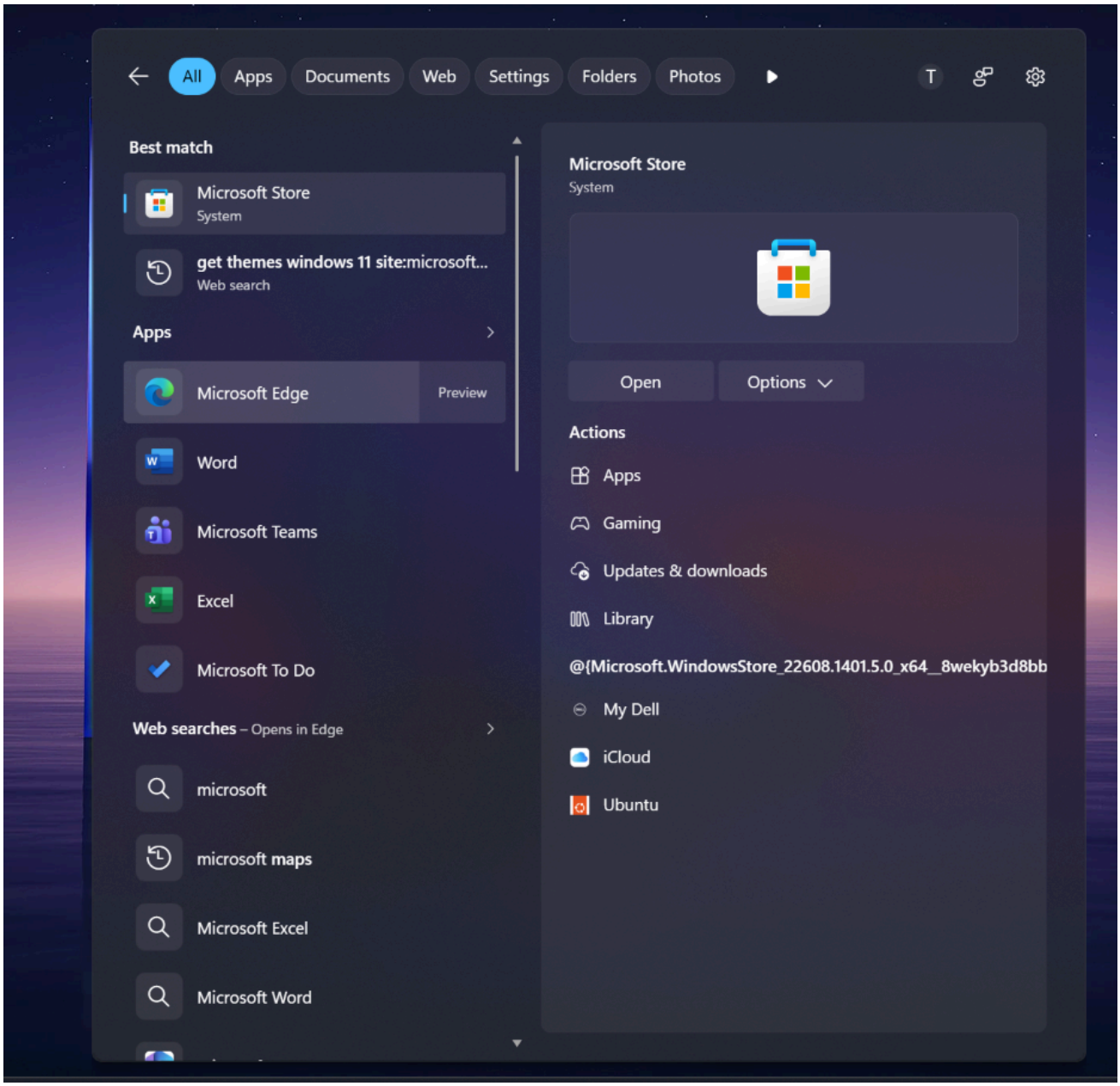


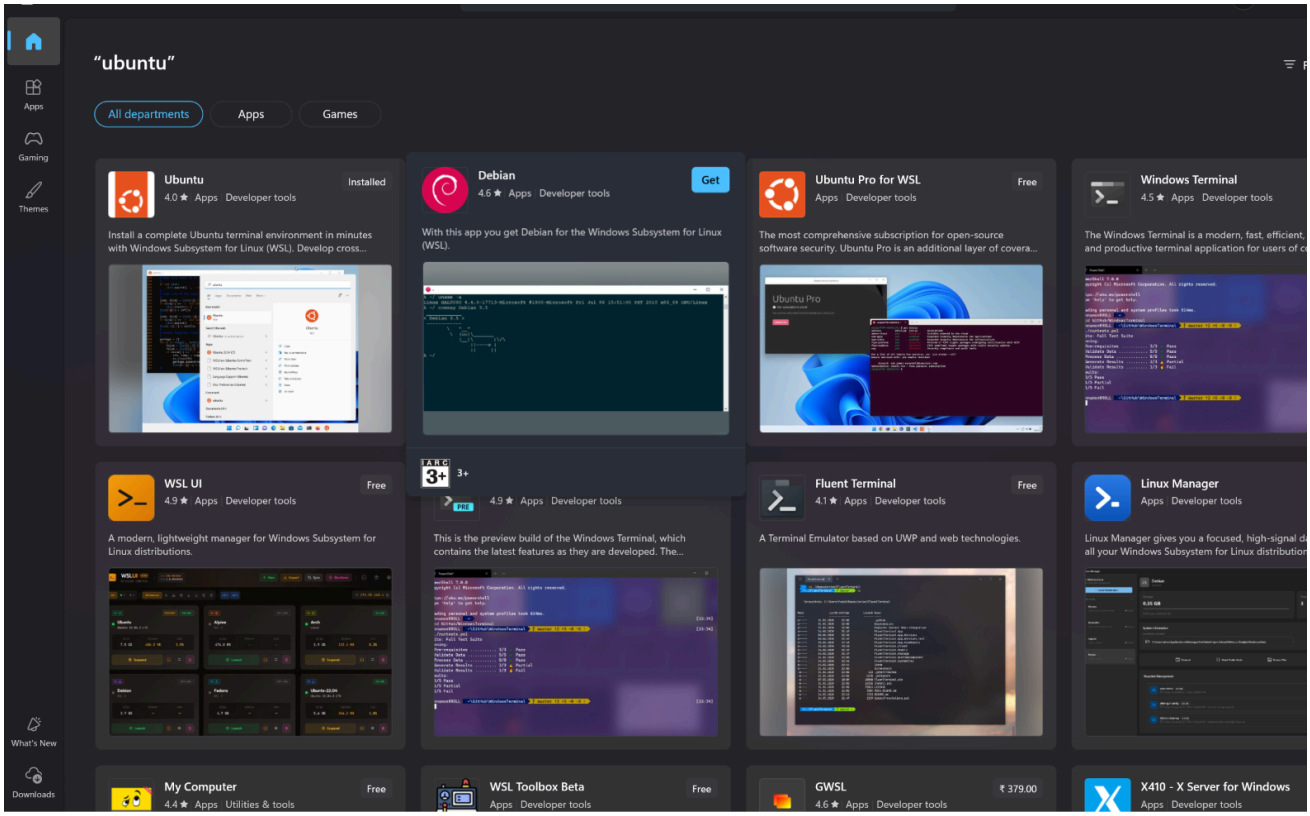


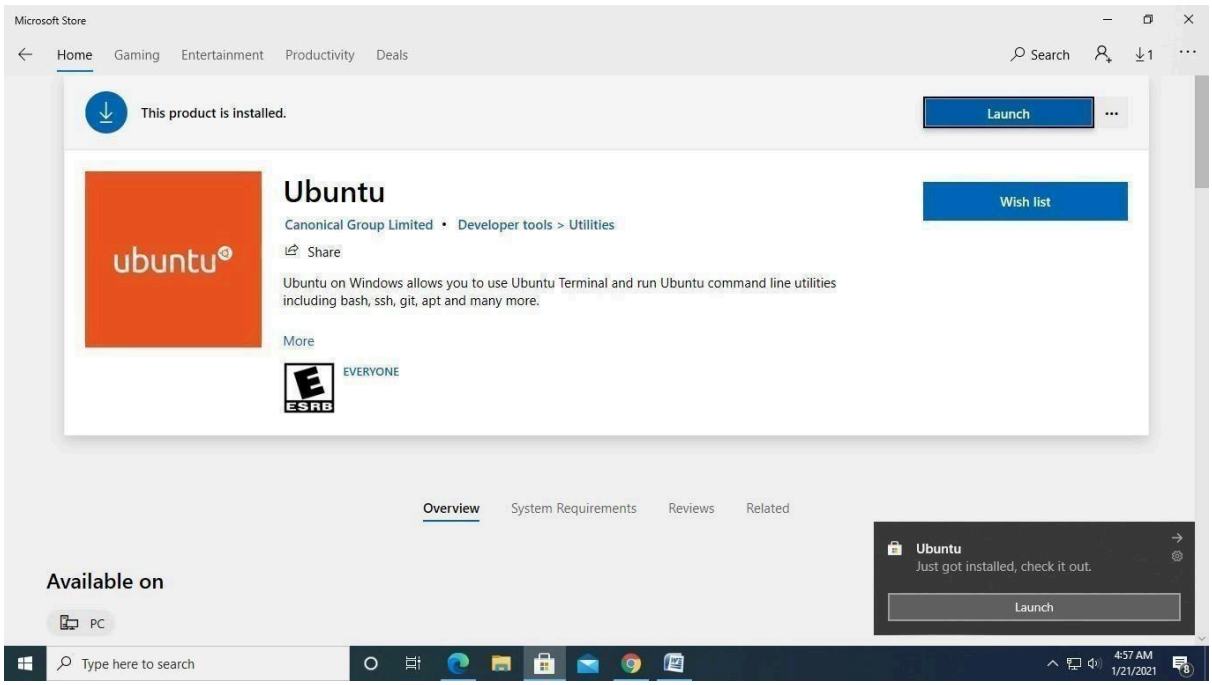
Select Turn features On or Off

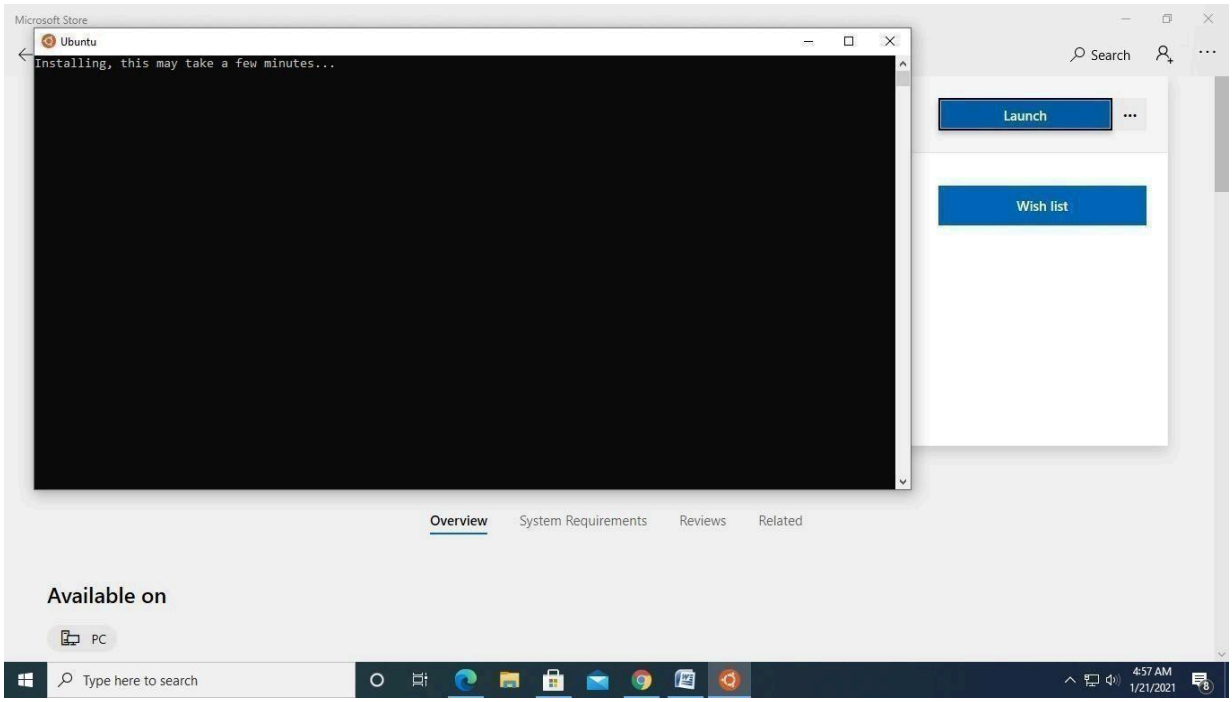



Recycle Bin
Document
OS
Programs
Control Panel
Search Control Panel
Control Panel Home
System and Security
Network and Internet
Hardware and Sound
Programs
User Accounts
Appearance and Personalization
Clock and Region
Ease of Access
Windows Features
Windows completed the requested changes.
Windows needs to reboot your PC to finish installing the requested changes.
Restart now
Don't restart
Type here to search
4:51 AM
1/21/2021

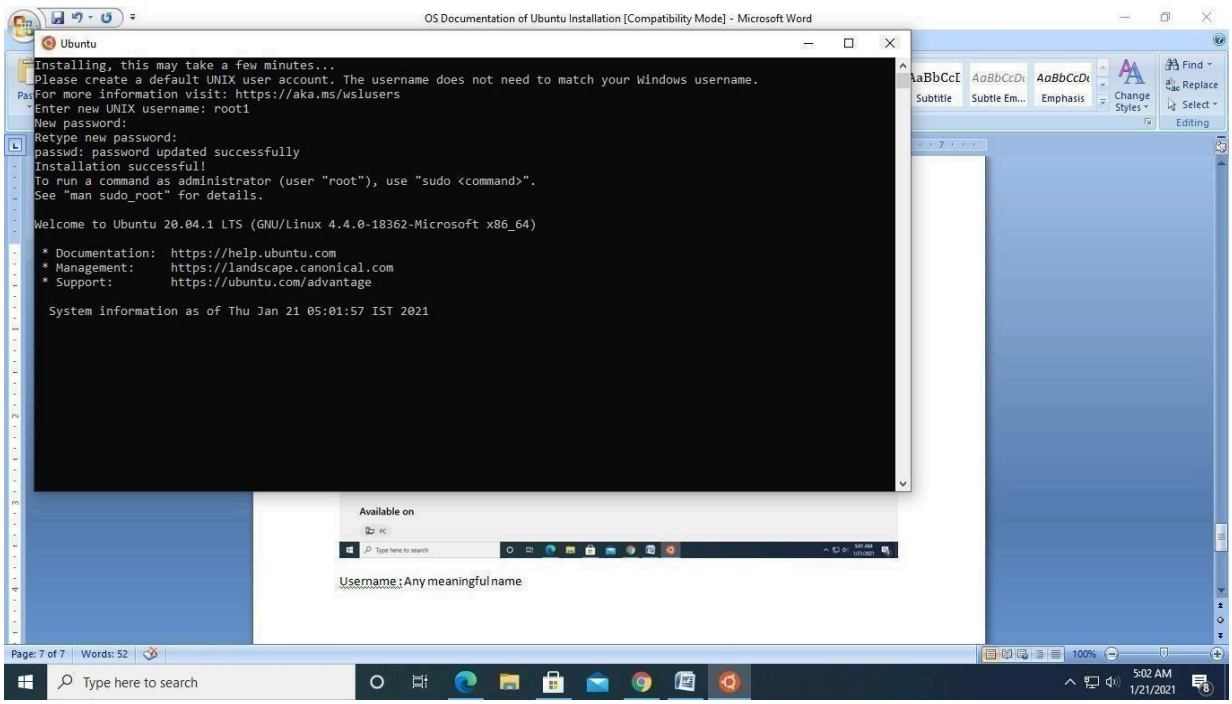

All
Apps
Documents
Web
Settings
Folders
Photos
Best match
Microsoft Store
System
get themes windows 11 site:microsoft...
Web search
Apps
Microsoft Edge
Preview
Word
Microsoft Teams
Excel
Microsoft To Do
Web searches – Opens in Edge
microsoft
microsoft maps
Microsoft Excel
Microsoft Word
Microsoft Store
System
Open
Options
Actions
Apps
Gaming
Updates & downloads
Library
@{Microsoft.WindowsStore_22608.1401.5.0_x64__8wekyb3d8bb
My Dell
iCloud
Ubuntu


"ubuntu"
Apps
Gaming
Themes
All departments
Apps
Games
Ubuntu
Installed
4.0 Apps Developer tools
Install a complete Ubuntu terminal environment in minutes with Windows Subsystem for Linux (WSL). Develop cross...
Debian
Get
4.6 Apps Developer tools
With this app you get Debian for the Windows Subsystem for Linux (WSL).
Ubuntu Pro for WSL
Free
Apps Developer tools
The most comprehensive subscription for open-source software security. Ubuntu Pro is an additional layer of covera...
Windows Terminal
4.5 Apps Developer tools
The Windows Terminal is a modern, fast, efficient, and productive terminal application for users of c
WSL UI
Free
4.9 Apps Developer tools
A modern, lightweight manager for Windows Subsystem for Linux distributions.
3+
4.9 Apps Developer tools
This is the preview build of the Windows Terminal, which contains the latest features as they are developed. The...
Fluent Terminal
Free
4.1 Apps Developer tools
A Terminal Emulator based on UWP and web technologies.
Linux Manager
Apps Developer tools
Linux Manager gives you a focused, high-signal d
all your Windows Subsystem for Linux distributio
What's New
Downloads
My Computer
Free
4.4 Apps Utilities & tools
WSL Toolbox Beta
Free
Apps Developer tools
GWSL
₹ 379.00
4.6 Apps Developer tools
X410 - X Server for Windows
Apps Developer tools


Microsoft Store
Home
Gaming
Entertainment
Productivity
Deals
Search
This product is installed.
Launch
Ubuntu
Canonical Group Limited • Developer tools > Utilities
Share
Wish list
Ubuntu on Windows allows you to use Ubuntu Terminal and run Ubuntu command line utilities including bash, ssh, git, apt and many more.
More
EVERYONE
Overview
System Requirements
Reviews
Related
Available on
PC
Ubuntu
Just got installed, check it out.
Launch
Type here to search
4:57 AM
1/21/2021



Username: Any meaningful name
Password: Any password



Finally, you will get the prompt on successful installation of Ubuntu in windows 10 64 bit

Program (4)	Output (3)	Viva (3)	Total (10)

Experiment.No-1B BASIC LINUX COMMANDS

a) Basic

1. echo SRM -To display the string SRM
2. clear - To clear the screen
3. date -To display the current date and time
4. Cal 2003 - to display the calender for the year

   2003 Cal 6 2003 - to display the calender for the

   june-2003
5. passwd -To change password

b) Working with Files

1.1s -list files in the present working directory

Is-1 -list files with detailed information (long list)

1s-a - list all files including the hidden files

2. cat> f1 - to create a file (Press^d to finish typing)
3. cat f1 display the content of the file f1
4. we fl - list no. of characters, words & lines of a file f1

   we -c f1 -list only no. of characters of files

   wc -w f1 -list only no. of words of file f1

   we -1fl -list only no. of lines of file fl1
5. cp f1 f2 - copy file f1 into f2
6. mv f1 f2 -rename file fl as f2
7. m f1 -remove the file f1
8. head-5 fl -list first 5 lines of the file fl

   tail-5 f1 - list last 5 lines of the file

   f1

c) Working with Directories

1. mkdir elias -to create the directory elias
2. cd elias - to change the directory as elias
3. mmdir elias - to remove the directory elias

4. pwd - to display the path of the present working directory

5. cd -to go to the home directory

cd.. - to go to the parent

directory

cd- -to go to the previous working

directory cd/ - to go to the root directory

d) File name substitution

1. Is f? - list files start with 'f and followed by any one character

2. ls *.c -list files with extension 'c

3. Is [gpylet - list files whose first letter is any one of the character g, p or y and followed by the word et

4. Is [a-d,l-m]ring -list files whose first letter is any one of the character froma to d andl to m and followed by the word ring.

e) I/O Redirection

1.Input redirection

we1<ex1 -To find the number of lines of the file 'ex1

2.Output redirection

who> f2 - the output of who' will be redirected to file f2

3. cat>> f1 - to append more into the file f1

f) Piping

Syntax: Command1 | command2

Output of the commandl is transferred to the command2 as input. Finally output of the command2 will be displayed on the monitor.

ex. cat f1 more list the contents of file f1 screen by

screen head-6 f1 tail-2 prints the 5th & 6th lines of the

file fl.

g) Environment variable

1. echo $HOME -display the path of the home directory

2. echo $PS1 - display the prompt string $

3. echo $PS2 -display the second prompt string (> symbol by default)

4. echo $LOGNAME -login name

5. echo $PATH -list of path mame where the OS searches for an executable file

h) File Permission

-- chmod command is used to change the access permission of a

file. Method-1

Syntax: chmod [ugo] [+/-] [rwxa] filename

u: user, g: group, o: others

+: Add permission-: Remove the permission

r: read, w: write, x: execute, a: all

permissions

ex. chmod ugtrw fl adding 'read & write' permissions of file f1 to bothuser

and group members.

Method-2

Syntax: chmod octrum filel

The 3 digit octal number represents as follows

- first digit -- file permissions for the user
- second digit -- file permissions for the group
-third digit -- file permissions for others

Each digit is specified as the sumof following

4-read permission, 2- write permission, 1- execute permission

ex. chmod 754 f1

it change the file permission for the file as follows

-read, write & execute pemmissions for the user ie; 4+2+1= 7

-read, & execute permissions for the group members ie; 4+0+1 = 5

-only read permmission for others ie; 4+0+0=4

Experiment.No-1C FILTERS and ADMIN COMMANDS FILTERS

1.cut

-Used to cut characters or fileds from a file/input

Syntax: cut -cchars filename

-ffieldnos filename

-By default, tabis the filed separator(delimiter). Ifthe fileds of the files are separated by any other character, we need to specify explicitly by-d option

cut-ddelimitchar-ffileds filname

2.Grep

- Used to search one or more files for a particular pattem

Syntax: grep pattem filename(s)

>Lines that contain the pattem in the file(s) get displayed

>pattem can be any regular expressions

>More than one files canbe searched for a pattem

-v option displays the lines that do not contain the patterm

-1 list only name ofthe files that contain the pattem

-n displays also the line number along with the lines that matches the patterm

3.Sort

-Used to sort the file in order

Syntax: sort filename

>Sorts the data as text by default

>Sorts by the first filed by default

-r option sorts the file in descending order

-u eliminates duplicate lines

-o filename writes sorted data into the file fname

-tdchar sorts the file in which fileds are separated by dchar

-n sorts the data as number

+ln skip first filed and sort the file by second filed numerically

4.Uniq

-Displays unique lines of a sorted file

Syntax: uniq filename

-d option displays only the duplicate lines

-c displays unique lines with no. of occurrences.

5.Diff

-Used to differentiate two files

Syntax: diff fl f2

compare two files fl & f2 and prints all the lines that are differed between f1 & f2.

Experiment.No.2 Process Creation using fork() and usage of getpid(), getppid(), wait() functions.

Aim: To write a program for process creation using fork() and usage of getpid(), getppid(), Wait() functions.

Defenations:

1. What is fork()?

fork() is a system call used to create a new child process.

2.What does getpid() do?

getpid() returns the PID of the current process.

3.What does getppid() do?

getppid() returns the PID of the parent process**.**

4.What is the use of wait()?

wait() makes the parent process wait until a child process terminates.

5.What does fork() return?

0 → Child process

Positive value → Parent process (child's PID)

-1 → fork() failed

Program:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int
main(){
fork();
fork();
printf("Hello World\n");
}
```

Output:

```
thiruni@THIRUNI:/mnt/c/Users/thiru$ nano os2a.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ gcc os2a.c -o os2a
thiruni@THIRUNI:/mnt/c/Users/thiru$ ./os2a
Hello World
Hello World
Hello World
thiruni@THIRUNI:/mnt/c/Users/thiru$ Hello World
```

Program:

- Usage of getpid() and
  getppid() #include <stdio.h>
  #include <sys/types.h>
  #include <unistd.h>
  void main()
  {
  //variable to store calling function's
  int process_id, p_ process_id;
  //getpid() - will return process id of calling function
  process_id = getpid();
  //getppid() - will return process id of parent function
  p_process_id = getppid();
  //printing the process ids
  printf("The process id: %d\n",process_id);
  printf("The process id of parent function: %d\n",p_process_id);

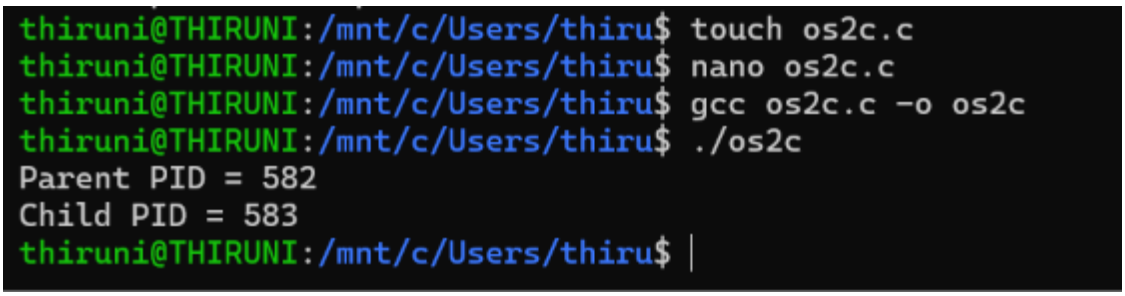
Output:

```
thiruni@THIRUNI:/mnt/c/Users/thiru$ touch os2b.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ nano os2b.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ gcc os2b.c -o os2b
thiruni@THIRUNI:/mnt/c/Users/thiru$ ./os2b
The process ID: 567
The parent process ID: 489
Child Process ID: 568
Parent Process ID: 567
Parent process completed.
thiruni@THIRUNI:/mnt/c/Users/thiru$
```

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h
>
#include<unistd.h>
int main()
{
pid_t cpid;
if (fork()== 0)
exit(0); /* terminate child */
else
cpid = wait(NULL); /* reaping parent */
printf("Parent pid = %d\n", getpid());
printf("Child pid = %d\n", cpid);
return 0;
}
```

Output:



Result:

Thus successfully completed process creation using fork() and usage of getpid(),wait() functions.

Program	Output	Viva	Total

Experiment.No.3 Multithreading and

Pthread in C Aim:

To implement and study multithreading and pthread in c

Defination:

1. What is a Thread?

A thread is the smallest unit of execution within a process. It is also called a lightweight process.

2. What is Single Threading?

Single threading means a process has only one thread, which executes one task at a time.

3. What is Multithreading?

Multithreading means a process has multiple threads that can execute multiple tasks concurrently.

4. What is Multithreading Using a Single Process?

It means multiple threads are created within the same process. The threads share the process's resources, such as memory.

Example: A browser using separate threads for downloading, displaying, and handling user input.

5. What is Multithreading Using Multiple Processes?

It means multiple processes are created, and each process can have one or more threads. Each process has its own separate memory space.

Example: Running a browser, music player, and text editor as separate processes.

Use Case: New Use Case: Smart Home Automation System

Part A – Single Process + Multi-Threading

An airport runway has three types of lights:

- Executes multiple appliance commands simultaneously without user lag
- Prevents total system failure if a single device thread crashes
- Optimizes memory usage on low-power home controllers

A smart home system manages different appliances using a single main controller. To ensure all devices activate simultaneously without waiting for each other, the system uses multiple threads.

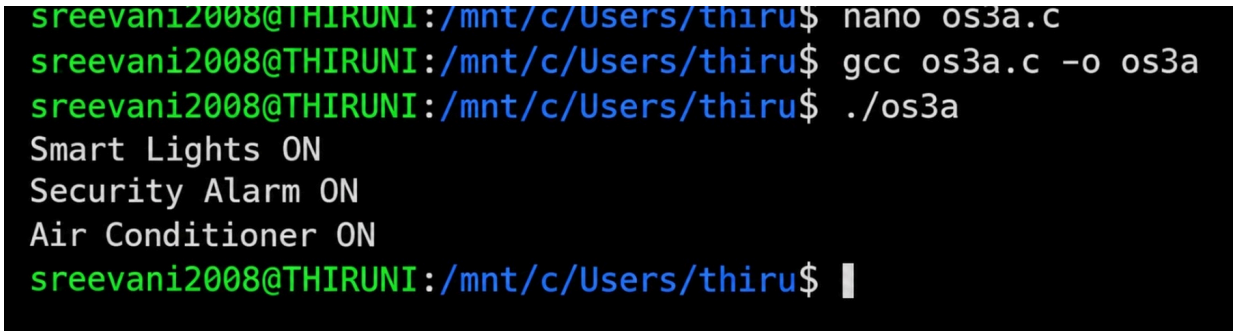
Programmatic Algorithm

1. Start the program.
2. Create three threads
3. Assign Thread 1 to control Smart Lights..
4. Assign Thread 2 to control the Air Conditioner.
5. Assign Thread 3 to control the Security Alarm..
6. Wait for all threads to complete execution using.
7. Stop the program.

Program :

```
#include<stdio.h>
#include<pthread.h>
void*e(void*x){printf("Smart Lights ON\n");}
void*c(void*x){printf("Air Conditions ON\n");}
void*w(void*x){printf("Security Alarm ON\n");}
int main(){pthread_t t1,t2,t3;
pthread_create(&t1,0,e,0); pthread_create(&t2,0,c,0); pthread_create(&t3,0,w,0);
pthread_join(t1,0); pthread_join(t2,0); pthread_join(t3,0);}
```

Output:



Part B – Multi-Process + Multi-Threading

Use Case

A large house has two independent controllers:

- Parent Process: handles the Living Room.
- Child Process: handles the Bedroom.

Each process creates its own thread to operate its assigned lights. This ensures that if one control unit fails, the other can continue functioning independently.

Programmatic Algorithm

1. Start the program.
2. Create a child process using fork().
3. Parent process creates a thread for Smart Lights.
4. Child process creates a thread for Air Conditions
5. Parent process also controls Security Alarm.
6. Wait for thread completion.
7. End the program.

Program :

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
void*bed(void*x) {printf("Bedroom Systems Active\n");}
void*liv(void*x) {prıntf("Living Room Systems Active\n");};
if(fork()==0){pthread_create(&t,0,run,0); pthread_join(t,0);}
pthread_create(&t,0,liv,0);
      pthread_join(t,0);
}     printf("Main House Alarm ON\n");
   }
```

Output:

```
gouse@LAPTOP-UHPMJ7JU:/mnt/c/Users/gouse$ touch ex3b.c
gouse@LAPTOP-UHPMJ7JU:/mnt/c/Users/gouse$ nano ex3b.c
gouse@LAPTOP-UHPMJ7JU:/mnt/c/Users/gouse$ gcc ex3b.c -o ex3b
gouse@LAPTOP-UHPMJ7JU:/mnt/c/Users/gouse$ ./ex3b
Living Room Systems Active
Bedroom Systems Active
Main House Alarm ON
gouse@LAPTOP-UHPMJ7JU:/mnt/c/Users/gouse$
```

Result:

Successfully implemented and studied multithreading and pthread in C

Program	Output	Viva	Total

Experiment.No.4 Mutual exclusion using semaphore and monitor

Aim:

To implement mutual exclusion using semaphore and monitor.

Defination:

1. What is Mutual?

Shared or held jointly by two or more processes.

2. What is Mutual Exclusion?

A rule in OS that prevents multiple threads from accessing a shared resource at the same time. Only one thread enters the critical section.

3. Functions & Disadvantages of Mutual Exclusion

POSIX Functions (C)

pthread_mutex_lock(): Locks the resource.

pthread_mutex_unlock(): Releases the resource.

Disadvantages

Deadlock: Threads block each other permanently.

Starvation: A thread waits forever to get the lock.

Overhead: Slows down CPU due to thread switching.

4. What are Semaphores and Monitors?

Semaphore: An integer variable used for synchronization using wait() and signal() operations.

Binary Semaphore: Value is 0 or 1.

Counting Semaphore: Value ranges from 0 to N.

Monitor: A high-level language feature that automatically handles locking so only one process runs inside it at a time.

5. Why Use Semaphores?

Resource Control: Allows up to N threads to access N resources (Mutex allows only 1).

Signaling: Lets threads notify each other when an event happens

Use Case: Electric Vehicle (EV) Charging Station

- Prevents hardware overload by restricting power draw to one vehicle.
- Eliminates system race conditions during simultaneous user requests..
- Enforces an orderly, predictable queue for waiting drivers..

Part A: Mutual Exclusion (Mutex)

Programmatic Algorithm

1. Start the program.
2. Initialize a mutex lock for the charging port..
3. Create two EV threads (Car A and Car B).
4. Lock the mutex before a car begins charging..
5. Display the charging status.
6. Unlock the mutex after charging is complete..
7. Wait for both threads to finish and destroy the mutex..
8. End the program.

Program :

```
#include<stdio.h>
#include<pthread.h>
pthread_mutex_t m;
void*ot(void*x){pthread_mutex_lock(&m);printf("EV
is Charging\n");pthread_mutex_unlock(&m);}
int main(){pthread_t t1,t2;pthread_mutex_init(&m,0);
pthread_create(&t1,0,ot,0);pthread_create(&t2,0,ot,0);
pthread_join(t1,0);pthread_join(t2,0);pthread_mutex_destroy(&m);
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os4a.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os4a.c -o os4a
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os4a
EV is Charging
EV is Charging
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Part B: Semaphore

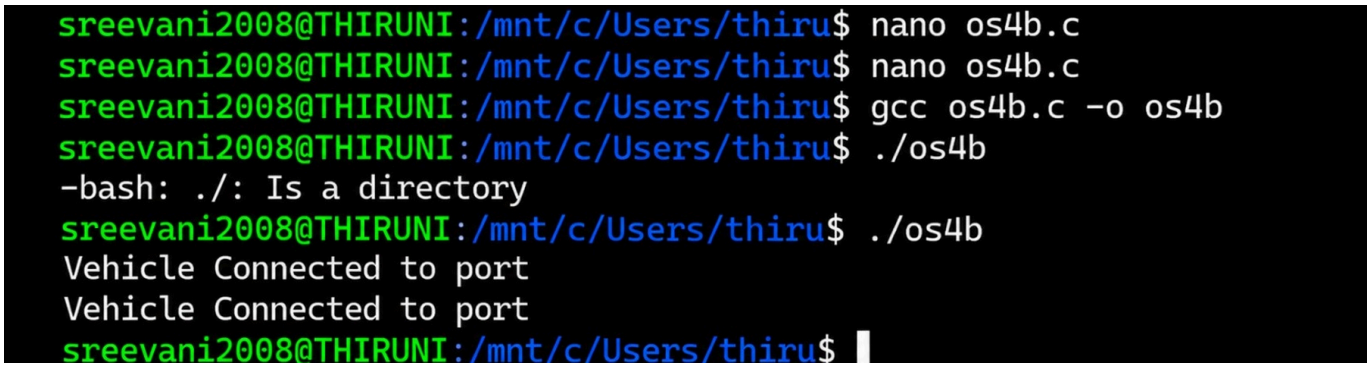
Programmatic Algorithm

1. Start the program.
2. Initialize a semaphore with value 1.
3. Create two EV threads.
4. Execute sem_wait before accessing the charging port.
5. Display that the vehicle has connected to the port..
6. Execute sem_post to signal that the port is free..
7. Wait for all threads.
8. End the program.

Program:

```
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
sem_t s;void*ot(void*x){sem_wait(&s);printf("Nurse Entered OT\n");sem_post(&s);}
int main(){pthread_t t1,t2;sem_init(&s,0,1);
pthread_create(&t1,0,ot,0);pthread_create(&t2,0,ot,0);
pthread_join(t1,0);pthread_join(t2,0);sem_destroy(&s);}
```

Output:



Part C: Mutual Exclusion using Semaphore and Monitor

Programmatic Algorithm

1. Start the program.
2. Initialize a semaphore.
3. Create a monitor function to control Charging Stationaccess.
4. EV threads request access strictly through the monitor function.
5 The monitor handles the wait and signal operations safely
6. .End the Program

.

.

Program:

```
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
sem_t s;void monitor(){sem_wait(&s);printf("Charging Station Access Granted\n");sem_post(&s);}
void*doc(void*x){monitor();}
int main(){pthread_t t1,t2;sem_init(&s,0,1);
pthread_create(&t1,0,doc,0);pthread_create(&t2,0,doc,0);
pthread_join(t1,0);pthread_join(t2,0);sem_destroy(&s);}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os4c.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os4c.c -o os4c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os4c
Charging station granted
Charging station granted
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Result:

Succesfully executed mutual exclusion using semaphore and monitor.

Program	Output	Viva	Total

Experiment.No.5 Reader-Writer problem

Aim:

To study the Reader-Writer problem

Definitions:

1. What is Reader and Writer in OS?

   Answer:

   - Reader: A process that only reads data from a shared resource.
   - Writer: A process that modifies or writes data to a shared resource.

2. Why do we use the Reader-Writer Problem?

   Answer:

   We use it to manage multiple processes accessing the same data and to prevent data inconsistency.

3. What are the defects (disadvantages) of the Reader-Writer Problem?

   Answer:

   - Starvation: A reader or writer may have to wait for a long time.
   - Deadlock: Processes may get stuck waiting for each other.
   - Complexity: Synchronization is difficult to implement correctly**.**

Use case: Blood-plasma storage temperature monitoring:

monitoring processes can read the temperature together, while an

alert/control process updates the temperature status when it

becomes abnormal.

Algorithm:

- Initialize a reader-writer lock and shared temperature.
- Create two reader threads and one writer thread.
- Readers acquire a read lock and display the temperature.
- Readers release the read lock.
- Writer acquires a write lock.
- Writer updates the temperature and prints an alert.
- Writer releases the write lock.

  Stop.

PROGRAM:

```
#include <stdio.h>
#include <pthread.h>

pthread_rwlock_t lock;
float temp = -25.0;

void *reader(void *p) {
   pthread_rwlock_rdlock(&lock);
   printf("Temperature: %.1f C\n", temp);
   pthread_rwlock_unlock(&lock);
   return NULL;
}

void *writer(void *p) {
   pthread_rwlock_wrlock(&lock);
   temp = -15.0;
   printf("Updated %.1f C - ALERT!\n", temp);
   pthread_rwlock_unlock(&lock);
   return NULL;
}

int main() {
   pthread_t r1, r2, w;
   pthread_rwlock_init(&lock, NULL);
   pthread_create(&r1, NULL, reader, NULL);
   pthread_create(&r2, NULL, reader, NULL);
   pthread_create(&w, NULL, writer, NULL);
   pthread_join(r1, NULL);
   pthread_join(r2, NULL);
   pthread_join(w, NULL);
   pthread_rwlock_destroy(&lock);
   return 0;

}
```

Output:

```
thiruni@THIRUNI:/mnt/c/Users/thiru$ touch os5.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ nano os5.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ gcc os5.c -o os5
thiruni@THIRUNI:/mnt/c/Users/thiru$ ./os5
Temperature: -25.0 C
Temperature: -25.0 C
Updated -15.0 C - ALERT!
thiruni@THIRUNI:/mnt/c/Users/thiru$
```

Result:

Thus successfully provided a solution to reader- writer using mutuex and semaphore.

Program	Output	Viva	Total

Experiment.N0.6 Dining philosopher's problem

Aim:

To implement and study Dining philosopher's problem

Defination

1. **What is the Dining Philosophers? Problem in OS**

Answer:

The Dining Philosophers Problem is a classic process synchronization problem in Operating Systems. It represents a situation where several processes (philosophers) share limited resources (forks) and need to use them without causing problems.

Each philosopher alternates between thinking and eating. To eat, a philosopher needs two forks, one on the left and one on the right. The problem is to design a system where philosophers can eat without deadlock or starvation.

2. **Why is the Dining Philosophers Problem used in OS?**

Answer:

The Dining Philosophers Problem is used to understand and demonstrate process synchronization and resource sharing in Operating Systems.

It helps us study:

- Deadlock – processes waiting for resources indefinitely.
- Starvation – a process waiting for a resource for an unlimited time.
- Mutual exclusion – ensuring that shared resources are used safely.
- Resource allocation – managing limited resources among multiple processes.
- Concurrency – handling multiple processes at the same time.

3. **What are the defects (limitations) of the Dining Philosophers Problem?**

Answer:

The main defects or problems are:

1. Deadlock: All philosophers may pick up one fork and wait forever for the other fork.
2. Starvation: A philosopher may repeatedly fail to get both forks and remain hungry.
3. Resource contention: Multiple philosophers compete for the same limited forks.
4. Complex synchronization: Proper synchronization is required to prevent conflicts.
5. Unfairness: Some philosophers may get opportunities to eat more frequently than others.

In short: The Dining Philosophers Problem demonstrates how improper resource management can lead to deadlock, starvation, and synchronization problems.

Use Case: Library Study Desk Access

- Philosophers: Students
- Shared Resources: Study desks
- Benefit: Ensures students get fair access to limited study desks without conflicts or indefinite waiting.

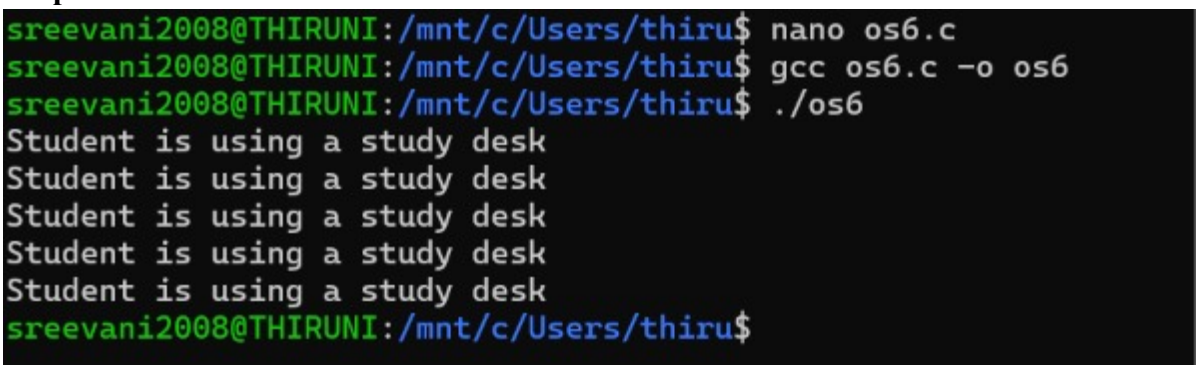
Programmatic Algorithm

1. Start.
2. Create five student threads.
3. Assign a study desk to each student.
4. Student requests access to a study desk.
5. Check whether the desk is available.
6. If available, allocate the desk.
7. Student studies for some time.
8. Release the desk after use.
9. Allow the next waiting student to access the desk.
10. Stop.

Program:

```
#include <stdio.h>
#include <pthread.h>
void *study(void *a){ printf("Student is using a study desk\n"); return NULL; }
int main(){
pthread_t s[5];
for(int i=0;i<5;i++)
pthread_create(&s[i],NULL,study,NULL);
for(int i=0;i<5;i++)
pthread_join(s[i],NULL);
return 0; }
```

Output:



Result:

Thus successfully implemented the concepts of Dining philosophers problem.

Program	Output	Viva	Total

Experiment.No.7 Bankers Algorithm for deadlock avoidance Aim:

To implement and study bankers algorithm for deadlock avoidance problem.

Definitions:

1. What is a Deadlock?

Answer:
A deadlock is a situation in an Operating System where two or more processes are permanently waiting for resources held by each other, so none of them can continue execution.

Example:

- Process P1 holds Resource R1 and waits for R2.
- Process P2 holds Resource R2 and waits for R1.
- Both processes keep waiting forever.

So, P1 → waits for R2 → held by P2
and P2 → waits for R1 → held by P1.

2. What are the causes of Deadlock?

A deadlock occurs when the following four conditions exist simultaneously:

1. Mutual Exclusion

A resource can be used by only one process at a time.

2. Hold and Wait

A process is holding one resource while waiting for another resource.

3. No Preemption

A resource cannot be forcibly taken away from a process. It must be released by the process itself.

4. Circular Wait

There is a circular chain of processes where each process is waiting for a resource held by the next process.

Example:
P1 → waits for P2 → waits for P3 → waits for P1.

3. What are the ways to avoid/manage Deadlock?

There are four common approaches:

1. Deadlock Prevention

Prevent at least one of the four deadlock conditions from occurring.

2. Deadlock Avoidance

Allocate resources only if doing so keeps the system in a

safe state. Algorithm used: Banker's Algorithm

3. Deadlock Detection

Allow deadlocks to occur and then detect them using a detection algorithm.

4. Deadlock Recovery

After detecting a deadlock, recover by:

- Terminating one or more processes.
- Preempting resources from processes

Use Case: Movie Ticket Booking & Cancellation System

Scenario

Consider an online movie-ticket system where multiple customers are booking or cancelling tickets simultaneously.

The system has limited resources such as:

- Seat Lock – temporarily reserves a movie seat.
- Payment/Refund Service – processes payment or refund.
- Cancellation Service – handles ticket cancellation.

Two processes may already hold one resource and request another. If resources are allocated carelessly, the processes can get stuck waiting for resources held by each other, resulting in deadlock.

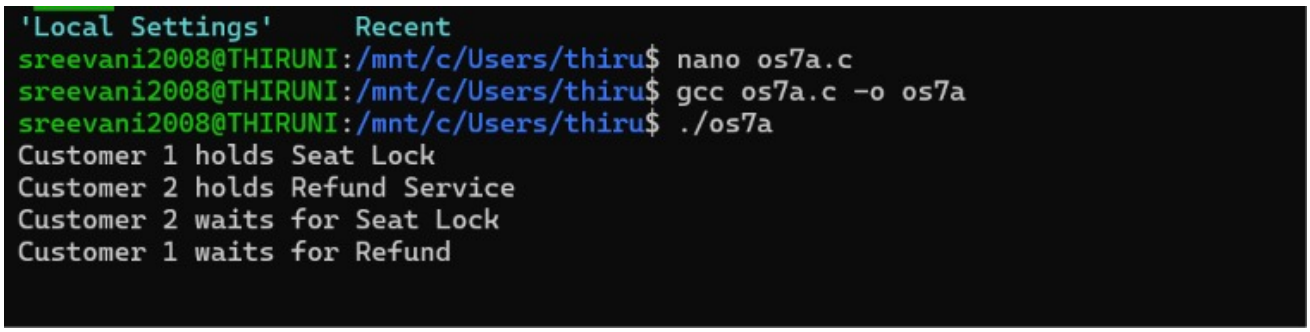
Programmatic Algorithm – Deadlock

1. Start.
2. Create two processes, Customer 1 and Customer 2.
3. Customer 1 acquires the Seat Lock and requests the Refund Service.
4. Customer 2 acquires the Refund Service and requests the Seat Lock.
5. Both resources are already held by the other process.
6. Customer 1 waits for Customer 2, and Customer 2 waits for Customer 1.
7. Neither process can proceed.
8. Deadlock occurs.
9. Stop.

Program:

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
pthread_mutex_t
seat=PTHREAD_MUTEX_INITIALIZER,refund=PTHREAD_MUTEX_INITIALIZER;
void* c1(void*x){pthread_mutex_lock(&seat);sleep(1);pthread_mutex_lock(&refund);}
void* c2(void*x){pthread_mutex_lock(&refund);sleep(1);pthread_mutex_lock(&seat);}
int main(){pthread_t
a,b;pthread_create(&a,0,c1,0);pthread_create(&b,0,c2,0);pthread_join(a,0);pthread_join(b,0);
}
```

Output:



Programmatic Algorithm – Banker's Algorithm

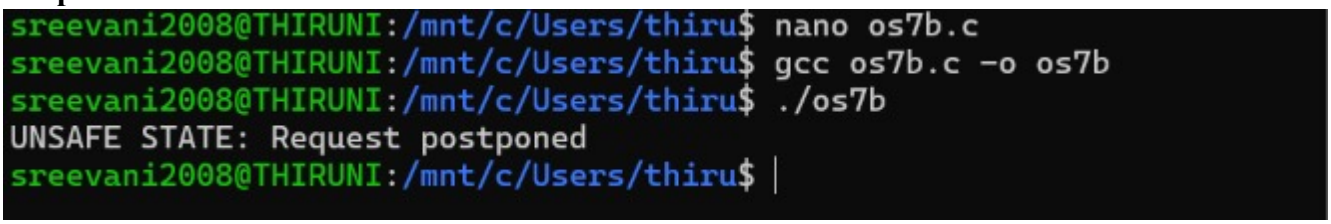
1. Start.
2. Define the available resources: Seat Lock and Refund Service.
3. Define the resources currently allocated to each customer.
4. Define the maximum resources each customer may need.
5. Calculate the Need = Maximum − Allocation.
6. Check whether the available resources can satisfy a customer's remaining need.
7. Temporarily allocate the resources.
8. Check whether all customers can complete their transactions.
9. If all customers can complete, the state is safe, so grant the request.
10. Otherwise, reject/postpone the request to avoid deadlock.
11. Stop.

Program:

```
#include<stdio.h>
int main(){
int avail=2,max[]={2,1},alloc[]={1,1},need[2],i;
for(i=0;i<2;i++) need[i]=max[i]-alloc[i];
printf("Movie Ticket Cancellation\n");
for(i=0;i<2;i++) if(need[i]<=avail){avail+=alloc[i];printf("Customer %d: Safe\n",i+1);}
else printf("Customer %d: Wait\n",i+1);
printf("System remains in SAFE STATE");

}
```

Output:



Result:

Successfully implemented the concepts of bankers algorithm.

Program	Output	Viva	Total

Experiment.No.8 FCFS and SJF

scheduling Aim:

To study the concept of FCFS and SJF schedulinig.

Definitions:

1. What is Scheduling?

Answer:
Scheduling is the process of selecting which process should be executed by the CPU and deciding the order in which processes will run.

The main goal of scheduling is to use the CPU efficiently and reduce waiting time and response time.

2. What is a Scheduling Algorithm?

Answer:

A scheduling algorithm is a set of rules used by the Operating System to decide which process should be executed next by the CPU.

Examples:

- FCFS (First Come First Serve)
- SJF (Shortest Job First)
- Round Robin
- Priority Scheduling

3. What is FCFS?

Answer:
FCFS (First Come First Serve) is a CPU scheduling algorithm in which the process that arrives first is executed first.

It follows the FIFO (First In, First Out) principle.

Example:

If processes arrive in this order:

P1 → P2 → P3

Then the execution order will be:

P1 → P2 → P3

Advantage: Simple and easy to implement.
Disadvantage: A long process can make shorter processes wait for a long time.

4. What is Shortest Job First (SJF)?

Answer:
Shortest Job First (SJF) is a CPU scheduling algorithm in which the process having the smallest CPU burst time is executed first.

Use case: In a bank, FCFS serves customers in arrival order, while SJF can serve small requests such as balance enquiries before longer loan-processing tasks

Part A: FCFS

Algorithm

1. Read number of processes and burst times.
2. Execute processes in arrival order.
3. First process waiting time is 0.
4. Next waiting time = previous waiting time + previous burst time.
5. Turnaround time = waiting time + burst time.
6. Display average waiting time and turnaround time.

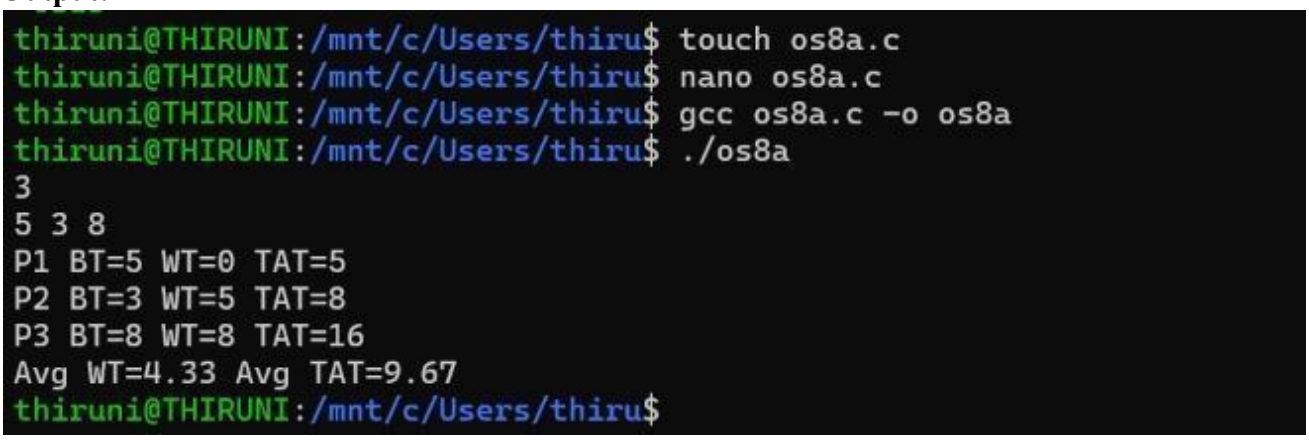
Program:

```
#include <stdio.h>

int main() {
   int n, i, bt[10], wt[10] = {0}, tat[10];
   float aw = 0, at = 0;
   scanf("%d", &n);
   for (i = 0; i < n; i++) scanf("%d", &bt[i]);
   for (i = 1; i < n; i++) wt[i] = wt[i - 1] + bt[i - 1];
   for (i = 0; i < n; i++) {
      tat[i] = wt[i] + bt[i];
      aw += wt[i];
      at += tat[i];
      printf("P%d BT=%d WT=%d TAT=%d\n", i + 1, bt[i], wt[i], tat[i]);
   }
   printf("Avg WT=%.2f Avg TAT=%.2f\n", aw / n, at / n);
   return 0;

}
```

Output:



Part B: SJF

Algorithm:

1. Read process burst times.
2. Sort processes in ascending order of burst time.
3. Execute the shortest process first.
4. Calculate waiting time and turnaround time.
5. Display the results and averages.

Program:

```
#include <stdio.h>

int main() {
   int bt[4] = {5, 3, 8, 2}, p[4] = {1, 2, 3, 4};
   int wt[4] = {0}, tat[4], i, j, t;
   float aw = 0, at = 0;
   for (i = 0; i < 3; i++)
      for (j = i + 1; j < 4; j++)
         if (bt[i] > bt[j]) {
            t = bt[i]; bt[i] = bt[j]; bt[j] = t;
            t = p[i]; p[i] = p[j]; p[j] = t;
         }
   for (i = 1; i < 4; i++) wt[i] = wt[i - 1] + bt[i - 1];
   for (i = 0; i < 4; i++) {
      tat[i] = wt[i] + bt[i];
      aw += wt[i];
      at += tat[i];
      printf("P%d BT=%d WT=%d TAT=%d\n", p[i], bt[i], wt[i], tat[i]);
   }
   printf("Avg WT=%.2f Avg TAT=%.2f\n", aw / 4, at / 4);
   return 0;

}
```

Output:



Result:

Implemented the concepts of FCFS and SJF scheduling in C succesfully.

Program	Output	Viva	Total

Experiment.No.9 Priority and Round Robin scheduling

Aim:

To study the concepts of priority and round robin scheduling

Definitions:

1. **What is Priority Scheduling?**

Answer:

Priority Scheduling is a CPU scheduling algorithm in which each process is assigned a priority, and the CPU is given to the process with the highest priority.

- A higher priority process is executed before a lower priority process.
- It can be preemptive or non-preemptive.
- If two processes have the same priority, FCFS can be used to decide which runs first.

Example:

If three processes have priorities:

- P1 → Priority 3
- P2 → Priority 1
- P3 → Priority 2

Then the execution order is P1 → P3 → P2 (assuming a larger number means higher priority).

2. **What is Round Robin Scheduling?**

Answer:

Round Robin Scheduling is a CPU scheduling algorithm in which each process gets a fixed amount of CPU time called a time quantum or time slice.

- Each process gets the CPU for one time quantum.
- If the process is not completed within that time, it is moved to the end of the ready queue.
- The next process gets the CPU.
- It is a preemptive scheduling algorithm.
- It is commonly used in time-sharing systems.

Use case: In a hospital emergency system, Priority scheduling handles the most urgent patient requests first. In a customer-support center, Round Robin gives every support ticket a fair fixed time slice.

Part A: Priority Scheduling

Algorithm

1. Read burst time and priority for each process.
2. Sort processes in descending order of priority.
3. Execute the highest-priority process first.
4. Calculate waiting time and turnaround time.
5. Display the scheduling order and averages.

Program:

```
#include <stdio.h>
int main() {
   int bt[4], pr[4], wt[4] = {0}, tat[4], i, j, t, x;
   for (i = 0; i < 4; i++)
      scanf("%d %d", &bt[i], &pr[i]);
   for (i = 0; i < 3; i++)
      for (j = i + 1; j < 4; j++)
         if (pr[i] < pr[j]) {
            x = pr[i]; pr[i] = pr[j]; pr[j] = x;
            x = bt[i]; bt[i] = bt[j]; bt[j] = x;
         }
   for (i = 0; i < 4; i++) {
      t += bt[i];
      tat[i] = t;
      wt[i] = tat[i] - bt[i];
      printf("P%d WT=%d TAT=%d\n", i + 1, wt[i], tat[i]);
   }
   return 0;
}
```

Output:

```
thiruni@THIRUNI:/mnt/c/Users/thiru$ touch os9a.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ nano os9a.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ gcc os9a.c -o os9a
thiruni@THIRUNI:/mnt/c/Users/thiru$ ./os9a
5 3

2 3
2 6
P1 WT=8 TAT=13
P2 WT=8 TAT=11
P3 WT=4 TAT=6
P4 WT=9 TAT=12
thiruni@THIRUNI:/mnt/c/Users/thiru$
```

Part B: Round Robin Scheduling

Algorithm

1. Read burst time of each process and the time quantum.
2. Copy burst times into remaining-time array.
3. Give each process CPU time equal to the quantum or its remaining time.
4. If a process is unfinished, keep it for the next round.
5. When a process finishes, calculate turnaround and waiting time.
6. Repeat until all processes finish.

Program:

```
#include <stdio.h>

int main() {

   int bt[4], r[4], tat[4], wt[4] = {0}, q, t = 0, c = 0, i, x;

   for (i = 0; i < 4; i++) {

      scanf("%d", &bt[i]);

      r[i] = bt[i];

   }

   scanf("%d", &q);

   while (c < 4)

      for (i = 0; i < 4; i++)

         if (r[i] > 0) {
```

```
            x = r[i] > q ? q : r[i];
            t += x;
            r[i] -= x;
            if (r[i] == 0) {
                tat[i] = t;
                wt[i] = tat[i] - bt[i];
                c++;
            }
        }
    for (i = 0; i < 4; i++)
        printf("P%d WT=%d TAT=%d\n", i + 1, wt[i], tat[i]);
    return 0;
}
```

Output:

```
thiruni@THIRUNI:/mnt/c/Users/thiru$ touch os9b.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ nano os9b.c
thiruni@THIRUNI:/mnt/c/Users/thiru$ gcc os9b.c -o os9b
thiruni@THIRUNI:/mnt/c/Users/thiru$ ./os9b
5 3 6 9
3
2
3
2
P1 WT=0 TAT=6
P2 WT=6 TAT=11
P3 WT=11 TAT=14
P4 WT=14 TAT=17
thiruni@THIRUNI:/mnt/c/Users/thiru$
```

Result:

Implemented the concepts of priority scheduling and round robin scheduling in c succesfully

Program	Output	Viva	Total

Experiment.No.10 FIFO Page Replaccement

Algorithm. Aim:

To implement FIFO page replacement algorithm.

Defanations:

1. What is FIFO?

Answer:

FIFO (First In, First Out) is a method in which the element or process that enters first is the first one to be removed or served.

It follows the principle that the first item added is the first item removed.

Example: Consider three customers standing in a queue:

- Customer 1 enters first.
- Customer 2 enters second.
- Customer 3 enters third.

The service order will be: Customer 1 → Customer 2 → Customer 3.

Advantages:

- Simple and easy to implement.
- Maintains the order of arrival.
- Prevents overtaking in a basic FIFO queue.

Applications in OS:

- FCFS (First Come, First Serve) CPU scheduling.
- Queue management.
- FIFO page replacement algorithm.

In short: FIFO means First In, First Out — whoever enters first is served first.

If you mean a **FIFO worker** as a job role, FIFO usually means **"Fly-In, Fly-Out."**

2.What does a FIFO worker do?

A FIFO worker travels to a remote work location for a fixed period, works there for several days or weeks, and then returns home for a period of rest.

For example:

14 days working → 7 days at home → 14 days working → 7 days at home

FIFO workers are commonly employed in:

- Mining

- Oil and gas
- Construction
- Remote infrastructure projects
- Engineering and maintenance

3.What does the worker actually do?

It depends on the job. For example, a FIFO **IT worker** might:

- Maintain computers and networks
- Troubleshoot technical problems
- Manage servers and systems
- Provide technical support

A FIFO engineer might work on equipment, construction, maintenance, or project operations.

Important: FIFO is a work arrangement, not a specific profession. The actual work depends on the person's job role.

Real-time use case: ATM Transaction System

In an ATM, several pages of a transaction program are loaded into limited memory. When a new page is required and memory is full, the FIFO (First-In-First-Out) Page Replacement algorithm removes the page that entered memory first.

Part A: FIFO

Programmatic Algorithm

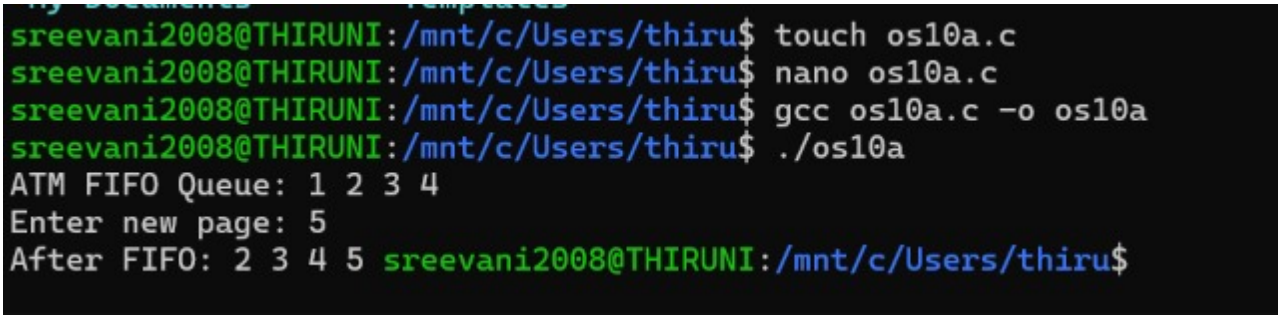
1. Start.
2. Create a queue of pages representing ATM transaction requests.
3. Insert each page into the queue in arrival order.
4. The first page inserted is considered the oldest.
5. When a new page arrives, check the queue.
6. If the page already exists, do not replace it.
7. If memory is full, remove the oldest page.
8. Insert the new page at the rear.
9. Display the FIFO page sequence.
10. Stop.

Program:

```
#include <stdio.h>
int main(){
 int q[4]={1,2,3,4}, i,p;
 printf("ATM FIFO Queue: ");
 for(i=0;i<4;i++) printf("%d ",q[i]);
 printf("\nEnter new page: "); scanf("%d",&p);
 for(i=0;i<3;i++) q[i]=q[i+1];
 q[3]=p;
```

```
 printf("After FIFO: ");
 for(i=0;i<4;i++) printf("%d ",q[i]);
 return 0;
}
```

Output:



Part B: Page Replacement

Programmatic Algorithm

1. Start.
2. Create a fixed-size memory containing ATM pages.
3. Read the requested page.
4. Search whether the page is already in memory.
5. If found, display Page Hit.
6. Otherwise, display Page Fault.
7. Replace the page occupying the selected frame.
8. Display the updated memory.
9. Continue for the required page requests.
10. Stop.

Program:

```
#include <stdio.h>
int main(){
 int m[3]={1,2,3},p,i,hit=0;
 printf("ATM Memory: 1 2 3\nEnter page: "); scanf("%d",&p);
 for(i=0;i<3;i++) if(m[i]==p) hit=1;
 if(hit) printf("Page Hit\n");
 else {m[0]=p; printf("Page Fault\n");}
 printf("Memory: ");
 for(i=0;i<3;i++) printf("%d ",m[i]);
 return 0;
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os10b.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os10b.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os10b.c -o os10b
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os10b
ATM Memory: 1 2 3
Enter page: 4
Page Fault
Memory: 4 2 3 sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Part C: FIFO + Page Replacement

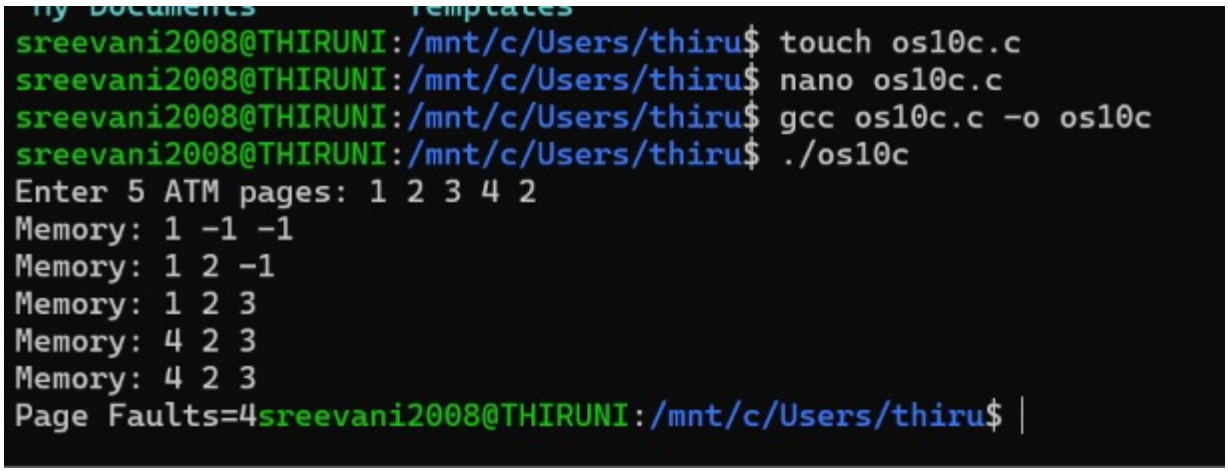
Programmatic Algorithm

1. Start.
2. Initialize ATM memory frames as empty.
3. Read the requested pages one by one.
4. Check whether the requested page is already in memory.
5. If present, count a Page Hit.
6. If absent, count a Page Fault.
7. If memory is full, replace the oldest page using FIFO.
8. Insert the new page into memory.
9. Display memory and total page faults.
10. Stop.

Program:

```
#include <stdio.h>
int main(){
 int f[3]={-1,-1,-1},r=0,p,i,j,hit, faults=0;
 printf("Enter 5 ATM pages: ");
 for(j=0;j<5;j++){scanf("%d",&p); hit=0;
  for(i=0;i<3;i++) if(f[i]==p) hit=1;
  if(!hit){f[r]=p;r=(r+1)%3;faults++;}
  printf("Memory: %d %d %d\n",f[0],f[1],f[2]);
 }
 printf("Page Faults=%d",faults); return 0;
}
```

Output:



Result:

Succesfully implemennted page replacement using FIFO algorithm.

Program	Output	Viva	Total

Experiment.No.11 LRU and LFU Page Replacement Algorithms.

Aim:

To implement page replacement using LRU and LFU

Definitions:

1. What is LRU?

Answer:
LRU (Least Recently Used) is a page replacement algorithm used in Operating Systems. It replaces the page that has not been used for the longest period of time.

Example:
If the pages in memory are:

1 2 3

and page 1 was used a long time ago while pages 2 and 3 were recently used, then page 1 will be replaced when a new page arrives.

In short:
LRU = Replace the page that was used least recently.

2. What is LFU?

Answer:
LFU (Least Frequently Used) is a page replacement algorithm used in Operating Systems. It replaces the page that has been used the fewest number of times.

Example:

Page	Number of times used
1	5
2	2
3	4

If a new page needs to be loaded, Page 2 will be replaced because it has been used the least number of times.

In short:
LFU = Replace the page that has been used the least frequently.

Difference between LRU and LFU

LRU	LFU
Least Recently Used	Least Frequently Used
Considers when a page was last used	Considers how many times a page was use
LRU	LFU
Replaces the page not used for the longest time	

Part A — LRU Page Replacement

Use Case

Smart Traffic Signal Camera – Least Recently Used Camera Page

When a new camera zone needs memory and all frames are occupied, the system replaces the camera page that has **not been accessed for the longest time**.

Programmatic Algorithm

1. Start.
2. Initialize three memory frames as empty.
3. Read the camera-zone page reference string.
4. Check whether the requested page is already in memory.
5. If present, update its recent-use time.
6. If absent, generate a page fault.
7. If an empty frame exists, load the page.
8. Otherwise, find the least recently used page.
9. Replace that page with the new camera page.
10. Display frames and total page faults.
11. Stop.

Program:

```
#include<stdio.h>
int main(){int f[3]={-1,-1,-1},t[3]={0},r[]={1,2,3,1,4,2,5},i,j,p,k=0,fault=0,m;
for(i=0;i<7;i++){p=r[i];for(j=0;j<3;j++)if(f[j]==p)break;
if(j==3){fault++;m=0;for(j=1;j<3;j++)if(t[j]<t[m])m=j;f[m]=p;}
for(j=0;j<3;j++)if(f[j]==p)t[j]=++k;
printf("Zone %d: ",p);for(j=0;j<3;j++)printf("%d ",f[j]);printf("\n");}
printf("Page Faults=%d\n",fault);return 0;
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os11a.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os11a.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os11a.c -o os11a
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os11a
Zone 1: 1 -1 -1
Zone 2: 1 2 -1
Zone 3: 1 2 3
Zone 1: 1 2 3
Zone 4: 1 4 3
Zone 2: 1 4 2
Zone 5: 5 4 2
Page Faults=6
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Part B — LFU Page Replacement

Use Case:

Smart Traffic Signal Camera – Least Frequently Used Camera Page

The traffic controller keeps frequently accessed camera zones in memory. When memory is full, the camera-data page with the lowest access frequency is replaced**.**

Programmatic Algorithm

1. Start.
2. Initialize three memory frames and frequency counters.
3. Read the camera-zone page reference string.
4. Check whether the requested page is present.
5. If present, increment its frequency.
6. If absent, generate a page fault.
7. If an empty frame exists, load the page.
8. Otherwise, find the page with the lowest frequency.
9. Replace it with the requested camera page.
10. Display frames and total page faults.
11. Stop.

Program:

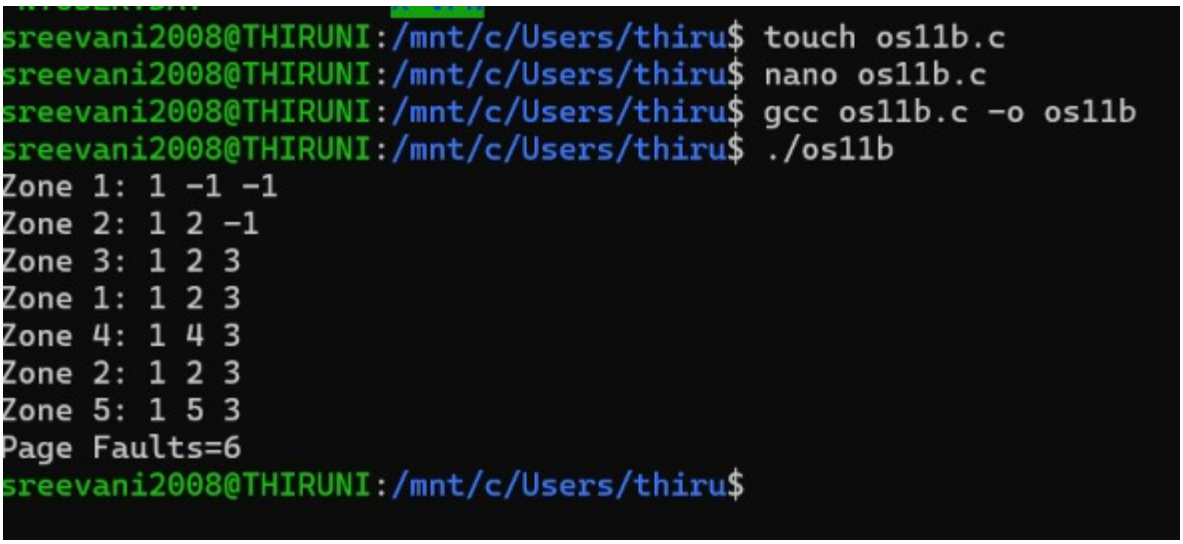
```
#include<stdio.h>

int main(){int f[3]={-1,-1,-1},c[3]={0},r[]={1,2,3,1,4,2,5},i,j,p,pos,min,fault=0;

for(i=0;i<7;i++){p=r[i];for(j=0;j<3;j++)if(f[j]==p)break;
```

```
if(j<3)c[j]++;else{fault++;pos=0;for(j=0;j<3;j++)if(f[j]==-1){pos=j;break;}
if(f[pos]!=-1){min=c[0];pos=0;for(j=1;j<3;j++)if(c[j]<min)min=c[j],pos=j;}f[pos]=p;c[pos]=1;}
printf("Zone %d: ",p);for(j=0;j<3;j++)printf("%d ",f[j]);printf("\n");}
printf("Page Faults=%d\n",fault);return 0;
}
```

Output:



Result:

Successfully implemented page replacement using LRU and LFU algorithms

Program	Output	Viva	Total

Experiment.No.12 Bestfit and Worstfit Memory Management Policies Aim:

To implement and study Best fit and worst fit memory management policies in C .

Definitions:

1. What is Best Fit?

Answer:

Best Fit is a memory allocation technique in Operating Systems. It allocates a process to the smallest available memory block that is large enough to hold the process.

Example:
Available blocks: 100 KB, 500 KB, 200 KB, 300 KB
Process size: 180 KB
Best Fit chooses 200 KB, because it is the smallest block that can accommodate 180 KB.

In short:

Best Fit → Choose the smallest suitable memory block.

2. What is Worst Fit?

Answer:

Worst Fit is a memory allocation technique that allocates a process to the largest available memory block.

Example:
Available blocks: 100 KB, 500 KB, 200 KB, 300 KB
Process size: 180 KB
Worst Fit chooses 500 KB, because it is the largest available block.

In short:

Worst Fit → Choose the largest available memory block.

Unique Real-Time Use Case:

Smart Drone Mission Memory Allocation System
A surveillance drone dynamically loads different mission modules such as Camera Processing, GPS Navigation, Obstacle Detection, and Thermal Imaging. The operating system must allocate available memory blocks to these modules efficiently.

Part A — Best Fit

Algorithm

1. Start.
2. Read the available memory blocks and process/module sizes.
3. For each module, search all available blocks.
4. Select the smallest block that is large enough for the module.
5. Allocate the module to that block.
6. Reduce the remaining size of the selected block.
7. Repeat for all modules.

8. Display the allocation.
9. Stop

Program:

```
#include <stdio.h>
int main(){
 int b[]={100,500,200,300,600},p[]={212,417,112,426},i,j,best;
 for(i=0;i<4;i++){best=-1;
  for(j=0;j<5;j++) if(b[j]>=p[i]&&(best==-1||b[j]<b[best])) best=j;
  if(best!=-1){printf("Module %d -> %dKB\n",p[i],b[best]);b[best]-=p[i];}
  else printf("Module %d -> Not Allocated\n",p[i]);
 }
}
```

Output:



Part B — Worst Fit

Algorithm

1. Start.
2. Read the memory blocks and module requests.
3. For each module, search all available blocks.
4. Select the largest available block that can accommodate the module.
5. Allocate the module to that block.
6. Reduce the remaining size of the selected block.
7. Repeat for all modules.
8. Display the allocation.
9. Stop.

Program:

```
#include <stdio.h>
int main(){
 int b[]={100,500,200,300,600},p[]={212,417,112,426},i,j,worst;
 for(i=0;i<4;i++){worst=-1;
  for(j=0;j<5;j++) if(b[j]>=p[i]&&(worst==-1||b[j]>b[worst])) worst=j;
  if(worst!=-1){printf("Module %d -> %dKB\n",p[i],b[worst]);b[worst]-=p[i];}
  else printf("Module %d -> Not Allocated\n",p[i]);
 }
}
```

Output:



Part C — Best Fit and Worst Fit

Algorithm

1. Start.
2. Initialize the same memory blocks and module requests.
3. Apply Best Fit and find the smallest suitable block.
4. Store its allocation results.
5. Reset the memory blocks.
6. Apply Worst Fit and find the largest suitable block.
7. Store its allocation results.
8. Compare both allocation strategies.

9. Display the results.
10. Stop.

Program:

```
#include <stdio.h>
void fit(int b[],int p[],int type){
 int i,j,k;
 for(i=0;i<4;i++){k=-1;
  for(j=0;j<5;j++) if(b[j]>=p[i]&&(k<0||(type?b[j]>b[k]:b[j]<b[k]))) k=j;
  if(k>=0){printf("%dKB -> %dKB\n",p[i],b[k]);b[k]-=p[i];}
  else printf("%dKB -> Not Allocated\n",p[i]);
 }
}
int main(){
 int b1[]={100,500,200,300,600},b2[]={100,500,200,300,600},p[]={212,417,112,426};
 printf("BEST FIT:\n");fit(b1,p,0); printf("WORST FIT:\n");fit(b2,p,1);
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os12c.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os12c.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os12c.c -o os12c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os12c
BEST FIT:
212KB -> 300KB
417KB -> 500KB
112KB -> 200KB
426KB -> 600KB
WORST FIT:
212KB -> 600KB
417KB -> 500KB
112KB -> 388KB
426KB -> Not Allocated
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Result:

Successfully implemented and studied best fit and worst fit memory management policies in C

Program	Output	Viva	Total

Experiment.No.13 Disk Scheduling Algorithm

Aim:

To implement and study disk scheduling algorithm

Definitions:

1. **What is Disk Scheduling?**

Answer:

Disk Scheduling is the method used by an Operating System to decide the order in which disk I/O requests are serviced.

It decides which request the disk head should handle first.

Examples: FCFS, SSTF, SCAN, C-SCAN, LOOK.

2. What is Disk Scheduling Needed For?

Answer:

Disk scheduling is needed to:

- Reduce seek time
- Reduce total head movement
- Improve disk performance
- Reduce waiting time
- Increase the number of requests processed per unit time (throughput)

3. **What is Seek Time?**

Answer:

Seek time is the time taken by the disk's read/write head to move from its current track to the required track.

Example:
If the head is at track 50 and the required track is 100, the head must move 50 tracks.

In simple words:
Seek time = Time required to move the disk head to the required track.

4. **What is Total Head Movement?**

Answer:

Total Head Movement is the total number of tracks traveled by the disk head while servicing all disk requests.

It is calculated by adding the distance moved between consecutive requests.

Example:

Head starts at 50 and requests are:

20 → 80 → 40

Total head movement:

|50 - 20| + |80 - 20| + |40 - 80|

= 30 + 60 + 40

= 130 tracks

In simple words:
Total Head Movement = Sum of all disk-head movements between requests.

Real-Life Use Case - Railway Platform CCTV System

A railway station has multiple CCTV cameras on different platforms that continuously generate video- recording storage requests. The SSTF disk scheduling algorithm selects the storage request closest to the current disk-head position, reducing unnecessary head movement and improving the efficiency of storing and accessing surveillance footage.

Programmatic Algorithm – Priority Scheduling

1. Initialize the number of processes, arrays for burst time, priority, waiting time, turnaround time, and set total_time = 0.
2. Input burst time and priority for each process.
3. Mark all processes as *unscheduled*.
4. Sort all processes by priority (lower number = higher priority).
5. For each process in sorted order:
    - Assign waiting time = current total_time.
    - Add burst time to total_time.
    - Calculate turnaround time = waiting time + burst time.
    - Mark the process as scheduled.
6. Repeat Step 5 until all processes are scheduled.
7. Display the complete execution sequence (process order).
8. Display the table of Burst Time, Priority, Waiting Time, and Turnaround Time.
9. Optionally, draw the Gantt chart to visualize execution.
10. End.

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main(){
 int n,h,r[20],v[20]={0},t=0,i,k,m,p;
 scanf("%d",&n);
 for(i=0;i<n;i++) scanf("%d",&r[i]);
 scanf("%d",&h); printf("%d",h);
 for(k=0;k<n;k++){m=9999;
  for(i=0;i<n;i++) if(!v[i]&&abs(r[i]-h)<m)m=abs(r[i]-h),p=i;
  v[p]=1;t+=m;h=r[p];printf(" -> %d",h);}
 printf("\nTotal Head Movement = %d\n",t);
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os13.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os13.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os13.c -o os13
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os13
5
82 170 43 140 24
50
50 -> 43 -> 24 -> 82 -> 140 -> 170
Total Head Movement = 172
sreevani2008@THIRUNI:/mnt/c/Users/thiru$
```

Result:

Successfully implemented various types of disk scheduling algorithm

Program	Output	Viva	Total

Experimentno:14 Sequential and Indexed file Allocation

Aim :
To implement and study Sequential and Indexed File Allocation

Definitions:

1. Sequential File Allocation
Sequential file allocation is a file allocation method in which all the blocks of a file are stored in consecutive (continuous) disk locations.
Example: If a file needs 4 blocks and starts at block 10, the blocks will be 10, 11, 12, 13.

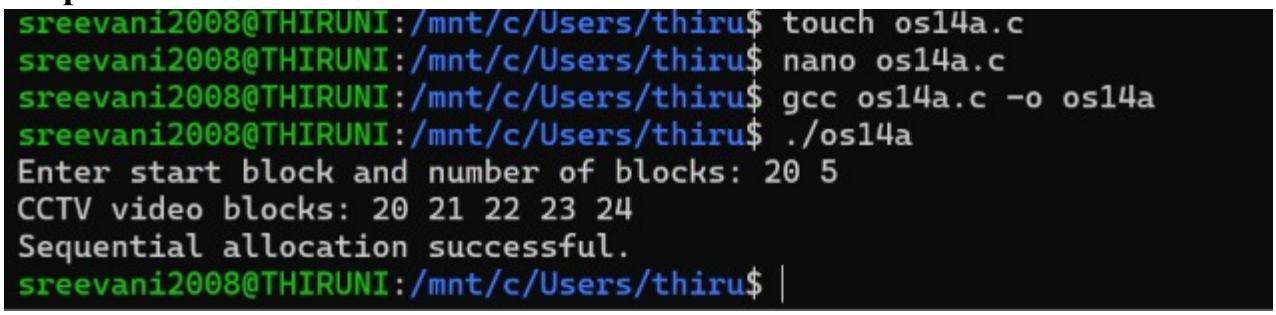
2.Indexed File Allocation
Indexed file allocation is a file allocation method in which an index block stores the addresses of all the disk blocks belonging to a file. The file blocks do not need to be consecutive.
Example: If a file uses blocks 5, 12, 20, 35, the index block stores these block numbers and helps locate the file data**.**

Real-Life Use Case – CCTV Video Recording
A CCTV surveillance system continuously records video footage. Sequential file allocation stores the video data in consecutive disk blocks, making sequential recording and playback faster and efficient.

Programmatic Algorithm
1. Read the starting block and number of blocks required.
2. Check the required number of consecutive blocks.
3. Allocate blocks starting from the given block.
4. Store each allocated block number.
5. Display all allocated blocks.
6. Continue until all required blocks are allocated.
7. Display the allocation status.

Program:
```
#include <stdio.h>
int main(){
 int start,n,i;
 printf("Enter start block and number of blocks: ");
 scanf("%d%d",&start,&n);
 printf("CCTV video blocks: ");
 for(i=0;i<n;i++) printf("%d ",start+i);
 printf("\nSequential allocation successful.\n");
 return 0;
}
```

Output:



Real-Life Use Case – Hospital Patient Records

A hospital information system stores patient records in different disk blocks. An index block stores the addresses of these blocks, allowing the system to quickly locate a patient's records.

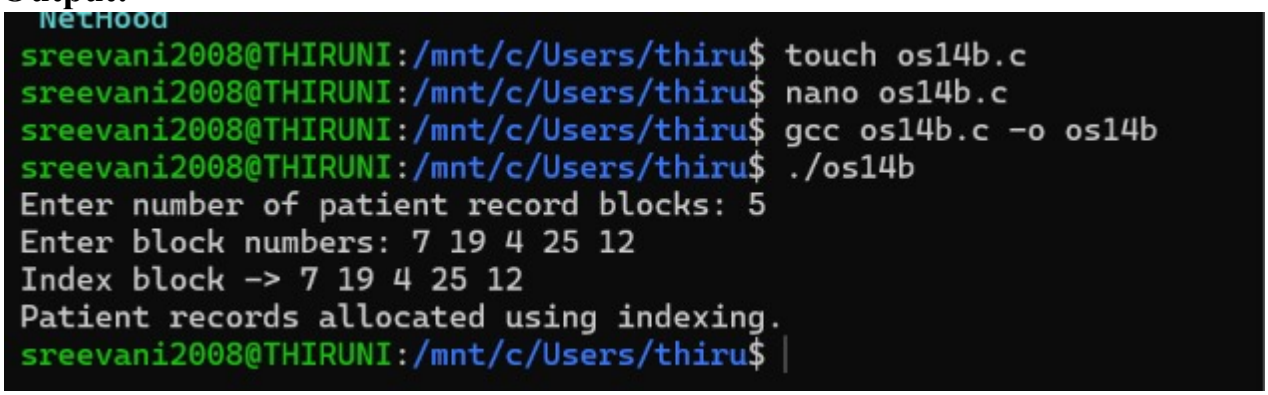
Programmatic Algorithm

1. Read the number of blocks required by the file.
2. Read the block numbers allocated to the file.
3. Store these block numbers in an index block.
4. Display the index block.
5. Display all referenced file blocks.
6. Use the index block to locate the file data.
7. Display the successful allocation.

Program:

```
#include <stdio.h>
int main(){
 int n,b[20],i;
 printf("Enter number of patient record blocks: ");
 scanf("%d",&n);
 printf("Enter block numbers: ");
 for(i=0;i<n;i++) scanf("%d",&b[i]);
 printf("Index block -> ");
 for(i=0;i<n;i++) printf("%d ",b[i]);
 printf("\nPatient records allocated using indexing.\n");
 return 0;
}
```

Output:



Result:

Successfully implemented and studied Sequential and Indexed File Allocation.

Program	Output	Viva	Total

Experimentno:15 File Organization Schemes for Single and Two Level Directory

Aim :

To implement and study file organization schemes for single and two level directory

Definitions:

1. Single-Level Directory

A Single-Level Directory is a file organization scheme in which all files are stored in one common directory. Each file must have a unique file name because there is only one directory.

Example:

College

```
├── OS.txt
├── DSA.txt
├── Java.txt
└── Python.txt
```

2. Two-Level Directory

A **Two-Level Directory** is a file organization scheme in which **each user has a separate directory** to store their files. Different users can have files with the **same name** because they are stored in different directories.

Example:

College

```
├── Student1
│   ├── OS.txt
│   └── DSA.txt
│
└── Student2
    ├── OS.txt
    └── Java.txt
```

Main difference:

- **Single-Level:** One common directory for all files.
- **Two-Level:** Separate directory for each user.

PART A — SINGLE-LEVEL DIRECTORY

Definition

A **Single-Level Directory** is a file organization scheme in which all files are stored in one common directory. Every file must have a unique name because there is only one directory.

Use Case :

College Student File Management:

A college stores assignment files of all students in a single common directory. The system allows the user to create and view student assignment files. Since all files are stored at the same level, duplicate file names are not allowed.

Programmatic Algorithm

1. Start and initialize the file array and set count n = 0.
2. Display the student file management menu.
3. Read the user's choice.
4. If the choice is **Create File**, enter the file name.
5. Check whether the file name already exists.

6. If it exists, display **File already exists**.
7. Otherwise, store the file name and increase n.
8. If **View Files** is selected, display all stored files.
9. Repeat the process until Exit is selected.
10. Stop..

Program:

```
#include <stdio.h>
#include <string.h>
int main(){
char f[10][30],name[30]; int n=0,ch,i,found;
do{
printf("\n-- COLLEGE FILE SYSTEM --\n1.Create File\n2.View Files\n3.Exit\nChoice: ");
scanf("%d",&ch);
if(ch==1){
printf("Enter file name: "); scanf("%s",name); found=0;
for(i=0;i<n;i++) if(strcmp(f[i],name)==0) found=1;
if(found) printf("File already exists!\n");
else {strcpy(f[n++],name); printf("File saved successfully.\n");}
}
else if(ch==2){for(i=0;i<n;i++) printf("%s\n",f[i]);}
}while(ch!=3);
return 0;
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os15a.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os15a.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os15a.c -o os15a
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os15a

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice: 1
Enter file name:  OS.txt
File saved successfully.

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice: 2
OS.txt

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice:
```

PART B — TWO-LEVEL DIRECTORY

Definition:

A Two-Level Directory is a file organization scheme in which each user has a separate directory for storing files. Different users can have files with the same name because their files are stored in different directories.

Use Case College Student File Management:

A college gives each student a separate directory to store assignment files. Each student can have files with the same name because the files are stored in different student directories.

Programmatic Algorithm:

1. Start and initialize two student directories.
2. Set the file count of each student to zero.
3. Display the student file management menu.
4. Read the user's choice.
5. If Create File is selected, select the student.
6. Enter the file name and store it in that student's directory.
7. If View Files is selected, select the student.
8. Display all files belonging to that student.
9. Repeat until Exit is selected.
10. Stop.

Program:

```
#include <stdio.h>
#include <string.h>
int main(){
char f[2][5][20],name[20]; int n[2]={0,0},s,ch,i;
do{
printf("\n-- COLLEGE FILE SYSTEM --\n1.Create File\n2.View Files\n3.Exit\nChoice: ");
scanf("%d",&ch);
if(ch==1){
printf("Enter student (1/2): "); scanf("%d",&s);
printf("Enter file name: "); scanf("%s",name);
strcpy(f[s-1][n[s-1]++],name); printf("File saved successfully.\n");
}
else if(ch==2){
printf("Enter student (1/2): "); scanf("%d",&s);
for(i=0;i<n[s-1];i++) printf("%s\n",f[s-1][i]);
}
}while(ch!=3);
return 0;
}
```

Output:

```
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ touch os15b
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ nano os15b.c
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ gcc os15b.c -o os15b
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os15.b
-bash: ./os15.b: No such file or directory
sreevani2008@THIRUNI:/mnt/c/Users/thiru$ ./os15b

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice: 1
Enter student (1/2): 1
Enter file name: OS.txt
File saved successfully.

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice: 2
Enter student (1/2): 1
OS.txt

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice: 1
Enter student (1/2): 2
Enter file name:  OS.txt
File saved successfully.

-- COLLEGE FILE SYSTEM --
1.Create File
2.View Files
3.Exit
Choice:
```

Result :
Successfully implemented file organization schemes for single level and two level directory.

Program	Output	Viva	Total